JSP

CHEAT SHEETS

A Complete Reference for JavaServer Pages, EL & JSTL

Directives · Implicit Objects · Scopes · Expression Language · Standard Actions · JSTL Core · fmt & fn · Tag Files · Tag Handlers · TLDs · MVC Data Flow · Security

34 REFERENCE SHEETS

CONTENTS

FOUNDATIONS & DIRECTIVES

01 JSP Overview & Request Lifecycle
02 Project Structure & Deployment
03 Page Directive
04 Scripting Elements
05 Include Directive
06 Taglib Directive

STANDARD ACTIONS

18 jsp:include & jsp:forward
19 jsp:useBean & Property Actions
20 Other Standard Actions

INTEGRATION & SECURITY

31 Servlet-to-JSP Data Flow
32 Error Handling & Security Output

QUICK REFERENCE

33 Quick Reference — Syntax, Objects & EL
34 Quick Reference — JSTL & Custom Tags

IMPLICIT OBJECTS & SCOPE

07 Implicit Objects — request & response
08 Implicit Objects — session & application
09 Implicit Objects — out, config, pageContext, page, exception
10 Page & Request Scope
11 Session & Application Scope
12 PageContext API & Scope Resolution

JSTL

21 JSTL Setup & Library Overview
22 JSTL Conditionals
23 JSTL Iteration
24 JSTL Output, Variables & URLs
25 JSTL Formatting (fmt:)
26 JSTL Functions (fn:)

EXPRESSION LANGUAGE

13 EL Syntax & Value Expressions
14 EL Implicit Objects
15 EL Operators & Type Coercion
16 EL Collection & Bean Access
17 EL Functions & Custom Functions

CUSTOM TAGS

27 Tag Files
28 Classic Tag Handlers
29 SimpleTag Handlers
30 Tag Library Descriptors (TLD)

JSP

Table of Contents

34 sheets · syntax · examples · notes

FOUNDATIONS & DIRECTIVES 01 – 06

- 01 JSP Overview & Reque…
- 02 Project Structure & De…
- 03 Page Directive
- 04 Scripting Elements
- 05 Include Directive
- 06 Taglib Directive

IMPLICIT OBJECTS & SCOPE 07 – 12

- 07 Implicit Objects — req…
- 08 Implicit Objects — ses…
- 09 Implicit Objects — out,…
- 10 Page & Request Scope
- 11 Session & Application …
- 12 PageContext API & Sc…

EXPRESSION LANGUAGE 13 – 17

- 13 EL Syntax & Value Expr…
- 14 EL Implicit Objects
- 15 EL Operators & Type C…
- 16 EL Collection & Bean A…
- 17 EL Functions & Custo…

STANDARD ACTIONS 18 – 20

- 18 jsp:include & jsp:forwa…
- 19 jsp:useBean & Propert…
- 20 Other Standard Actions

JSTL 21 – 26

- 21 JSTL Setup & Library …
- 22 JSTL Conditionals
- 23 JSTL Iteration
- 24 JSTL Output, Variables…
- 25 JSTL Formatting (fmt:)
- 26 JSTL Functions (fn:)

CUSTOM TAGS 27 – 30

- 27 Tag Files
- 28 Classic Tag Handlers
- 29 SimpleTag Handlers
- 30 Tag Library Descriptor…

INTEGRATION & SECURITY 31 – 32

- 31 Servlet-to-JSP Data Fl…
- 32 Error Handling & Secur…

QUICK REFERENCE 33 – 34

- 33 Quick Reference — Sy…
- 34 Quick Reference — JST…

Sheet 01

JSP Overview & Request Lifecycle

_jspService · translation phase · servlet container · JSP vs Servlet · first-request latency

WHAT JSP IS

A **JavaServer Page** is a text file (`.jsp`) that mixes static HTML/XML template text with JSP syntax — directives, scripting elements, and action tags. The servlet container **translates** it into a Java servlet class, **compiles** that class, and then executes it on every request. You write the template; the container writes the servlet.

KEY FACTS

- File extension is `.jsp`; fragment includes use `.jspf`
- Translation happens *once* per change — subsequent requests hit compiled bytecode
- JSP 3.1 is part of Jakarta EE 10; package prefix changed from `javax.*` to `jakarta.*`
- JSP's job is the **view** — generating response output, not business logic
- Every JSP compiles to a class that extends `HttpJspBase`, which extends `HttpServlet`

LIFECYCLE — WHAT THE CONTAINER DOES

```
// ── PHASE 1: Translation (once, on first request or redeploy)
//   Container reads hello.jsp
//   → generates hello_jsp.java (a servlet class)

// ── PHASE 2: Compilation (once)
//   hello_jsp.java → hello_jsp.class

// ── PHASE 3: Initialization (once per class load)
void jspInit() { /* override in <%! %> declaration */ }

// ── PHASE 4: Request handling (every request)
void _jspService(HttpServletRequest  request,
                 HttpServletResponse response)
    throws ServletException, IOException {
    // Your JSP template becomes code here.
    // You CANNOT override _jspService directly.
}

// ── PHASE 5: Destruction (on undeploy / shutdown)
void jspDestroy() { /* override in <%! %> declaration */ }
```

LIFECYCLE NOTES

- → `Translation` — only reruns when the `.jsp` file changes (or server restarts)
- → `Compilation` — produces a `.class` in the container's work directory
- → `jspInit()` — declare in `<%! %>`; called once after class load
- → `_jspService()` — called per request; template output goes here; cannot be overridden
- → `jspDestroy()` — declare in `<%! %>`; clean up resources here
- → First request is slow (translate + compile); all subsequent requests are fast

MODERN EQUIVALENT

- Spring MVC + Thymeleaf: templates rendered by a controller, no generated servlet
- React / Next: rendering moves to JS

PREFER / AVOID

✗ ~~logic-heavy JSPs~~ → ✓ **servlet / controller + a thin view**

✗ ~~first-request compile in prod~~ → ✓ **precompile JSPs**

JSP VS SERVLET — RESPONSIBILITY SPLIT

CONCERN	SERVLET	JSP
Request parsing	✔	—
Business logic	✔	—
Data retrieval	✔	—
HTML output / view	—	✔
Error pages	✔	✔
URL mapping	✔	—

JSP SYNTAX CATEGORIES

CATEGORY	SYNTAX
Directive	`<%@ ... %>`
Scriptlet	`<% Java code %>`
Expression	`<%= value %>`
Declaration	`<%! member %>`
EL Expression	`${ expr }`
Standard Action	`<jsp:... />`
Custom Tag	`<prefix:tag />`
JSP Comment	`<%-- text --%>`

WHAT GETS GENERATED

JSP SOURCE	GENERATED JAVA
`<html>Hi</html>`	out.write("<html>Hi</html>")
`<%= name %>`	out.print(name)
`<% int x=1; %>`	int x=1; (inline in _jspService)
`<%! int n; %>`	int n; (class-level field)
`${user.name}`	EL evaluation call
`<%-- comment --%>`	(nothing — removed)

⚠ COMMON MISTAKES

- Putting business logic in JSP — the container translates it into servlet code, but it still belongs in the servlet or service layer
- Expecting `jspInit()` / `jspDestroy()` to work without declaring them in `<%! %>` — they must be declarations, not scriptlets
- Trying to override `_jspService()` — this is generated by the container; any attempt is silently ignored or causes a compile error
- Mixing `javax.servlet` and `jakarta.servlet` imports — pick one based on your Jakarta EE version and stick to it

✓ TIPS

- Pre-compile JSPs at build time with the Maven `tomcat7-maven-plugin` or `jetty-jspc-maven-plugin` to catch errors before deployment
- The container's work directory (e.g. `Tomcat/work/`) holds the generated `.java` files — read them when debugging unexpected output
- Use `<%@ page trimDirectiveWhitespaces="true" %>` to strip whitespace-only lines introduced by directives from the response
- Tag the JSP version explicitly: `<%@ page language="java" %>` is the only valid language value — makes intent clear

QUICK SUMMARY

- `.jsp` — text file translated into a servlet by the container
- `translation + compile` — happens once; requests hit bytecode after
- `_jspService()` — generated method that handles every request; not overridable
- `jspInit() / jspDestroy()` — lifecycle hooks; declare in `<%! %>`
- `JSP = view` — servlet handles logic, JSP renders output

Sheet 02

Project Structure & Deployment

WAR layout · WEB-INF · URL mapping · webapps dir · precompilation · hot reload

WAR STRUCTURE

A JSP web application is packaged as a **WAR** (Web Application Archive). The WAR has a mandatory layout: anything under `WEB-INF/` is *never directly accessible* by the browser — only the server can read those files. JSPs placed at the webapp root or in subdirectories outside `WEB-INF/` are URL-accessible. JSPs inside `WEB-INF/jsp/` can only be reached via a `RequestDispatcher` forward from a servlet.

KEY FACTS

- `WEB-INF/` contents are never served directly — protected by the container
- Best practice: store all JSPs under `WEB-INF/jsp/` and dispatch to them from servlets
- Tomcat's work directory holds generated `.java` and `.class` files — inspect when debugging
- Dropping a WAR into `tomcat/webapps/` auto-deploys it; the context path equals the WAR filename
- `ROOT.war` deploys to the root context `/`

STANDARD WAR LAYOUT

```
myapp.war
├── index.jsp                  # URL: /myapp/
├── css/
│   └── style.css
├── js/
│   └── app.js
└── WEB-INF/
    ├── web.xml                # Deployment descriptor
    ├── classes/               # Compiled .class files
    │   └── com/example/
    │       └── MyServlet.class
    ├── lib/                   # JAR dependencies (JSTL, etc.)
    │   ├── jakarta.servlet.jsp.jstl-3.0.1.jar
    │   └── jakarta.servlet.jsp.jstl-api-3.0.0.jar
    └── jsp/                   # Protected JSPs (recommended)
        ├── home.jsp
        ├── login.jsp
        └── error.jsp

# Maven project source layout mirrors this:
src/main/webapp/               # = WAR root
src/main/webapp/WEB-INF/
src/main/webapp/WEB-INF/web.xml
```

LAYOUT NOTES

- → `WEB-INF/web.xml` — deployment descriptor; servlet mappings, error pages, session config
- → `WEB-INF/classes/` — servlet and bean `.class` files; added to container classpath automatically
- → `WEB-INF/lib/` — JAR files; scanned for TLD files; on classpath automatically
- → `WEB-INF/jsp/` — convention, not required; keeps JSPs browser-inaccessible
- → `index.jsp` at root — served as welcome file if listed in `web.xml`

MODERN EQUIVALENT

- Spring Boot: executable JAR, templates in `src/main/resources/templates`

PREFER / AVOID

- ✗ ~~JSPs in the web root~~ → ✓ **under `WEB-INF/` (not directly reachable)**
- ✗ ~~hot-deploying WARs in prod~~ → ✓ **immutable builds**

SECURITY CHECK

- JSPs outside `WEB-INF` can be requested directly
- Keep views in `WEB-INF/views`, reach them via a controller

URL → FILE MAPPING

URL	RESOLVED FILE	BROWSER DIRECT?
`/myapp/`	index.jsp (welcome)	✔
`/myapp/about.jsp`	about.jsp at root	✔
`/myapp/home`	Servlet → forward to WEB-INF/jsp/home.jsp	✖
`/myapp/WEB-INF/…`	Blocked by container	✖

TOMCAT DEPLOYMENT

ACTION	HOW
Deploy WAR	Copy `myapp.war` → `webapps/`
Context path	WAR filename = context (e.g. `/myapp`)
Root context	Name the WAR `ROOT.war`
Hot reload	Replace WAR; Tomcat auto-expands
Precompile JSPs	Use `jspc` or Maven plugin at build time
Work directory	`work/Catalina/localhost/myapp/`
Logs	`logs/catalina.out`, `localhost.log`

MAVEN POM.XML ESSENTIALS

ELEMENT	VALUE
`<packaging>`	war
`jsp-api scope`	provided
`servlet-api scope`	provided
`jstl scope`	compile (runtime)
`Plugin`	maven-war-plugin
`Source webapp`	src/main/webapp

⚠ COMMON MISTAKES

- Placing JSPs at the WAR root when they should be protected — a user can request `/myapp/checkout.jsp` directly and bypass the servlet controller
- Adding `jakarta.servlet.jsp-api` as `compile` scope — the container provides it; use `provided` or the JAR conflicts at runtime
- Forgetting to include JSTL JARs in `WEB-INF/lib/` — the container doesn't bundle JSTL; a missing taglib URI causes a translation error
- Editing files inside the expanded WAR in `webapps/` — changes are lost on next redeploy; always edit source and rebuild

✓ TIPS

- Always store view JSPs under `WEB-INF/jsp/` — forces every request through a servlet, preventing direct URL access to templates
- Use the Tomcat Manager app (`/manager/html`) to redeploy individual WARs without restarting the server
- Inspect the generated servlet in `work/Catalina/localhost/<app>/org/apache/jsp/` when tracking down output or compilation bugs
- Set `<reloadable>true</reloadable>` in `context.xml` during development for automatic class reload — disable in production

QUICK SUMMARY

- `WEB-INF/` — container-protected; never browser-accessible
- `WEB-INF/lib/` — JARs auto-added to classpath; put JSTL here
- `WEB-INF/jsp/` — recommended home for all view JSPs
- `ROOT.war` — deploys to root context `/`
- `work/` dir — holds generated `.java` ; inspect when debugging

Page Directive

WHAT THE PAGE DIRECTIVE DOES

The **page directive** (`<%@ page %>`) configures the JSP page itself — it controls how the container translates and executes the page. It is processed at *translation time*, not at request time, so its attributes affect the generated servlet class, not individual responses. A page can have multiple page directives, but each attribute may only appear once (except `import`).

KEY FACTS

- Processed at translation time — changing an attribute requires a file touch or server restart to take effect
- `import` is the only attribute that can appear multiple times; all others are once-per-page
- Place the page directive at the very top of the file, before any output, to avoid accidental whitespace in the response headers
- Default `contentType` is `text/html;charset=ISO-8859-1` — always override to UTF-8
- EL is enabled by default in JSP 2.0+; set `isELIgnored="true"` only for legacy pages

COMMON PAGE DIRECTIVE PATTERNS

```
<%-- Minimal recommended header for every JSP --%>
<%@ page language="java"
         contentType="text/html; charset=UTF-8"
         pageEncoding="UTF-8" %>

<%-- Import one or more Java classes --%>
<%@ page import="java.util.List, java.util.ArrayList" %>
<%@ page import="com.example.model.User" %>  <%-- can repeat --%>

<%-- Disable session creation (read-only pages) --%>
<%@ page session="false" %>

<%-- Designate an error page --%>
<%@ page errorPage="/WEB-INF/jsp/error.jsp" %>

<%-- Mark a page as the error handler --%>
<%@ page isErrorPage="true" %>

<%-- Control response buffering --%>
<%@ page buffer="16kb" autoFlush="true" %>

<%-- Disable EL (legacy JSP 1.x pages only) --%>
<%@ page isELIgnored="true" %>

<%-- Strip whitespace from directive lines --%>
<%@ page trimDirectiveWhitespaces="true" %>
```

ATTRIBUTE NOTES

- → `language` — always `"java"`; only valid value
- → `contentType` — sets `Content-Type` response header; always set UTF-8
- → `pageEncoding` — tells the container how to read the *source file*; set to match your editor
- → `import` — generates Java `import` statements; comma-separated or repeated
- → `session="false"` — prevents `HttpSession` creation; removes `session` implicit object
- → `errorPage` — URL to forward to on unhandled exception (must be app-relative)
- → `isErrorPage="true"` — enables the `exception` implicit object on this page
- → `buffer` — size of output buffer; `"none"` disables buffering

MODERN EQUIVALENT

- Thymeleaf: no directives — content type and errors come from Spring config

PREFER / AVOID

- ✗ ~~`import` of many classes~~ → ✓ **no Java in the page**
- ✗ ~~`session="true"` by default~~ → ✓ `session="false"` **where not needed**

SECURITY CHECK

- `errorPage` must not print stack traces
- Generic error pages; log details server-side

ALL PAGE DIRECTIVE ATTRIBUTES

ATTRIBUTE	DEFAULT	DESCRIPTION
`language`	"java"	Scripting language (only java)
`contentType`	text/html; charset=ISO-8859-1	Response Content-Type header
`pageEncoding`	ISO-8859-1	Source file character encoding
`import`	(none)	Java import; repeatable
`session`	true	Create/use HttpSession
`buffer`	8kb	Output buffer size or "none"
`autoFlush`	true	Flush when buffer full
`isThreadSafe`	true	Deprecated — do not use
`errorPage`	(none)	URL of error handler JSP
`isErrorPage`	false	Enable exception implicit object
`isELIgnored`	false	Treat ${} as literal text
`trimDirectiveWhitespaces`	false	Remove whitespace-only directive lines

⚠ COMMON MISTAKES

- Omitting `pageEncoding="UTF-8"` and `contentType="text/html; charset=UTF-8"` — the default ISO-8859-1 breaks non-ASCII characters silently
- Using `errorPage` with an absolute URL starting with `http://` — it must be a context-relative path starting with `/`
- Setting `autoFlush="false"` without increasing `buffer` — a buffer overflow throws an `IOException` instead of flushing
- Forgetting to set `isErrorPage="true"` on the error page itself — without it, the `exception` implicit object is null

✓ TIPS

- Create a shared header fragment with the standard page directive and include it in every JSP with `<%@ include file="header.jspf" %>`
- Use `trimDirectiveWhitespaces="true"` on pages that return JSON or XML — directive whitespace pollutes the response body
- Prefer `<error-page>` in `web.xml` over per-page `errorPage` attribute — one central declaration covers all JSPs
- Never use `isThreadSafe="false"` — it implements `SingleThreadModel`, which is deprecated and causes severe performance problems

QUICK SUMMARY

- `<%@ page %>` — configures translation; processed once, not per-request
- `import` — only repeatable attribute; adds Java imports
- `contentType + pageEncoding` — always set both to UTF-8
- `errorPage / isErrorPage` — wire JSPs to an error handler
- `session="false"` — skip session for stateless/read-only pages

Scripting Elements

<% %> · <%= %> · <%! %> · generated servlet mapping · thread safety

THE THREE SCRIPTING ELEMENT TYPES

JSP provides three scripting delimiters for embedding Java directly in a page. **Scriptlets** (`<% %>`) inject statements into `_jspService()`. **Expressions** (`<%= %>`) evaluate and print a value — no semicolon. **Declarations** (`<%! %>`) add class-level members outside `_jspService()`, making them shared across all requests.

KEY FACTS

- Scriptlet variables are *local* to `_jspService()` — they disappear after the response
- Declaration variables are *instance fields* — shared across all concurrent requests (thread-unsafe without synchronization)
- `<%= expr %>` calls `out.print(expr)` — do not add a semicolon inside
- HTML between scriptlets is emitted as `out.write("...")` in the generated servlet
- Modern JSP: prefer EL `${}` and JSTL over scriptlets for all new code

SCRIPTING ELEMENT SYNTAX

```
<%-- SCRIPTLET: Java statements → inside _jspService() --%>
<%
    String user = request.getParameter("name");
    if (user == null) user = "Guest";
%>
<p>Hello, <%= user %></p>   <%-- EXPRESSION: prints user --%>

<%-- Loop mixing Java and HTML --%>
<%
    List<String> items = (List<String>) request.getAttribute("items");
    for (String item : items) {
%>
    <li><%= item %></li>
<% } %>

<%-- DECLARATION: class-level field and method --%>
<%!
    private int hitCount = 0;          // shared — not thread-safe alone

    private String greet(String name) {
        return "Hello, " + name + "!";
    }
%>
<p><%= greet(user) %></p>
<p>Hits: <%= ++hitCount %></p>         <%-- race condition! --%>

<%-- JSP COMMENT: stripped at translation — never in response --%>
<%-- This differs from <!-- HTML comment --> which IS sent --%>
```

GENERATED SERVLET MAPPING

- → `<% stmt; %>` → statement placed inline in `_jspService()`
- → `<%= expr %>` → `out.print(expr)` in `_jspService()`
- → `<%! member %>` → class-level field or method, outside `_jspService()`
- → `HTML text` → `out.write("HTML text")` in `_jspService()`
- → `<%-- --%>` → removed entirely; not even a Java comment remains
- → `${ }` → EL evaluation call; preferred over `<%= %>`

MODERN EQUIVALENT

- Thymeleaf: `th:text`, `th:if` — no Java in templates
- JSX expressions in React

PREFER / AVOID

- ✗ ~~`<% %>` scriptlets~~ → ✓ **JSTL + EL**
- ✗ ~~`<%! %>` fields (shared across threads)~~ → ✓ **request attributes**

SECURITY CHECK

- `<%= request.getParameter("q") %>` = reflected XSS
- Output through `<c:out>`

THREE-WAY COMPARISON

ELEMENT	SYNTAX	WHERE GENERATED	SEMICOLON?
Scriptlet	`<% stmt %>`	Inside _jspService()	✔ required
Expression	`<%= expr %>`	out.print() in _jspService()	✖ forbidden
Declaration	`<%! member %>`	Class body (instance level)	✔ required
JSP Comment	`<%-- --%>`	Removed entirely	—

VARIABLE SCOPE & THREAD SAFETY

DECLARED IN	SCOPE	THREAD SAFE?
`<% %> scriptlet`	Local to _jspService()	✔ yes
`<%! %> declaration`	Instance field (shared)	✖ no
`request.setAttribute`	Request scope	✔ yes
`session.setAttribute`	Session scope	✖ sync needed

⚠ COMMON MISTAKES

- Adding a semicolon inside `<%= expr; %>` — this is a compilation error; expressions must not end with `;`
- Using `<%! %>` for mutable counters or state — instance fields are shared across all concurrent requests without synchronization
- Expecting HTML comments `<!-- -->` to hide sensitive output — they are sent to the browser; use JSP comments `<%-- --%>` instead
- Declaring loop variables in `<%! %>` instead of `<% %>` — they become shared instance fields, causing data corruption under concurrent load

✓ TIPS

- Replace `<%= bean.getName() %>` with `${bean.name}` — EL is shorter, null-safe, and auto-escapes in JSTL `<c:out>`
- When you must use scriptlets, keep them to a single variable assignment or cast — any logic beyond that belongs in the servlet
- Use `<%! %>` only for constants (`static final`) and `jspInit()` / `jspDestroy()` lifecycle hooks
- Read the generated servlet in `work/` to understand exactly what your scriptlet becomes — it demystifies output ordering bugs

QUICK SUMMARY

- `<% %>` — scriptlet; Java statements in _jspService(); local scope
- `<%= %>` — expression; prints value; no semicolon
- `<%! %>` — declaration; class-level; shared across requests
- `<%-- --%>` — JSP comment; never sent to browser
- `${}` — prefer EL over expressions for all output

Include Directive

STATIC INCLUDE AT TRANSLATION TIME

The **include directive** (`<%@ include file="..." %>`) copies the content of another file *verbatim* into the current JSP at **translation time**, before any Java is compiled. The result is a single merged servlet. Variables declared in the included file are visible in the including page because they become part of the same Java method. This is ideal for shared headers, footers, and navigation fragments.

KEY FACTS

- Inclusion happens at *translation time* — the included file is pasted in before compilation
- Fragment files conventionally use `.jspf` extension (JSP Fragment) to signal they are not standalone pages
- The `file` attribute is always a *context-relative* path (starts with `/`) or page-relative path — no runtime expression allowed
- Variables declared in included fragments are visible to the including page — they merge into the same `_jspService()`
- If the fragment changes, the including page must be re-translated — touch the parent file or restart

INCLUDE DIRECTIVE USAGE

```
<%-- page-relative path --%>
<%@ include file="header.jspf" %>

<%-- context-relative path (starts with /) --%>
<%@ include file="/WEB-INF/fragments/nav.jspf" %>

<%-- The included fragment can use variables from the parent
     because they merge into the same _jspService() --%>

<%-- header.jspf — a typical fragment --%>
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>${pageTitle}</title>        <%-- EL works in fragments --%>
    <link rel="stylesheet" href="${pageContext.request.contextPath}/css/style.css">
</head>
<body>

<%-- footer.jspf --%>
<footer>© 2024 My App</footer>
</body>
</html>

<%-- home.jsp — uses both fragments --%>
<%@ page pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/fragments/header.jspf" %>
<main>
    <h1>Welcome</h1>
</main>
<%@ include file="/WEB-INF/fragments/footer.jspf" %>
```

CODE NOTES

- → `file` — path resolved at translation time; no runtime expressions, no query strings
- → `.jspf` — conventional extension; signals "not a standalone page"; configure your IDE accordingly
- → Fragment inherits the parent's implicit objects — `request`, `session`, `out` etc. are all available
- → Avoid duplicate `<%@ page %>` directives across parent + fragment — same attribute twice is a translation error
- → `${pageContext.request.contextPath}` — use this for absolute links inside fragments so paths work regardless of URL depth

MODERN EQUIVALENT

- Thymeleaf: `th:insert` / `th:replace` fragments

PREFER / AVOID

- ✗ ~~static include for dynamic parts~~ → ✓ `jsp:include`
- ✗ ~~`.jsp`~~ ~~partials~~ → ✓ `.jspf` **fragments under** `WEB-INF`

INCLUDE DIRECTIVE VS JSP:INCLUDE ACTION

FEATURE	<%@ INCLUDE %>	<JSP:INCLUDE>
When resolved	Translation time	Request time
Dynamic path	✖	✔
Shared variables	✔	✖
Separate servlet	✖	✔
Change detection	Manual (touch file)	Automatic
Best for	Static fragments (nav, footer)	Dynamic content

PATH RESOLUTION RULES

PATH FORM	RESOLVED FROM	EXAMPLE
Starts with /	Context root	/WEB-INF/frags/nav.jspf
No leading /	Same directory as including page	header.jspf
../	Parent directory (page-relative)	../common/nav.jspf
Query string	❌ Not allowed — translation time only	

⚠ COMMON MISTAKES

- Declaring `contentType` in both the parent page and the included fragment — duplicate page directive attributes cause a translation error
- Expecting changes to a fragment to take effect without re-translating the parent — the container only watches the top-level `.jsp` file for changes in many configurations
- Using a runtime EL expression in the `file` attribute — `file="${path}"` is illegal; use `<jsp:include page="${path}">` instead
- Putting a `<!doctype>` declaration in every included fragment — it ends up duplicated in the response when multiple fragments are included

✓ TIPS

- Store all fragments under `WEB-INF/fragments/` using `.jspf` extension — keeps them protected and clearly distinguished from full pages
- Use `${pageContext.request.contextPath}` for all absolute resource paths in fragments — works correctly regardless of which page includes the fragment
- Extract the `<%@ page contentType ... %>` directive into only one file (the top-level page) — fragments should omit it entirely
- Configure your server to auto-detect changes to included files: set `modificationTestInterval=0` in Tomcat's `context.xml` during development

QUICK SUMMARY

- `<%@ include file="..." %>` — pastes file content at translation time
- `.jspf` — conventional extension for non-standalone fragments
- Shared scope — included file sees parent's variables; they become one servlet
- No runtime paths — `file` must be a literal string, not an expression
- Use `<jsp:include>` instead when the path is dynamic or changes often

Taglib Directive

DECLARING A TAG LIBRARY

The **taglib directive** (`<%@ taglib %>`) imports a tag library into the current JSP page and assigns it a *prefix*. After the directive, tags from that library are available as `<prefix:tagname>`. The `uri` attribute is a logical identifier — not a literal URL — that the container resolves against TLD files found in `WEB-INF/` or inside JARs in `WEB-INF/lib/`.

KEY FACTS

- The `uri` is a *logical name*, not a network address — the container looks it up in TLD files
- TLDs are discovered automatically from JARs in `WEB-INF/lib/` and files in `WEB-INF/`
- Prefix is arbitrary but conventional — use `c:` for JSTL core, `fmt:` for formatting
- A page can declare multiple taglibs with different prefixes
- Jakarta EE 10 changed URIs from `http://java.sun.com/jsp/jstl/…` to `jakarta.tags.*`

TAGLIB DIRECTIVE USAGE

```
<%-- ── Jakarta EE 10 / JSTL 3.x URIs (current) ──────────── --%>
<%@ taglib prefix="c"   uri="jakarta.tags.core" %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>
<%@ taglib prefix="fn"  uri="jakarta.tags.functions" %>
<%@ taglib prefix="sql" uri="jakarta.tags.sql" %>
<%@ taglib prefix="x"   uri="jakarta.tags.xml" %>

<%-- ── Legacy javax / JSTL 1.x URIs (pre-Jakarta EE 9) ─── --%>
<%@ taglib prefix="c"   uri="http://java.sun.com/jsp/jstl/core" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
<%@ taglib prefix="fn"  uri="http://java.sun.com/jsp/jstl/functions" %>

<%-- ── Custom tag library (TLD in WEB-INF/) ──────────────── --%>
<%@ taglib prefix="my"  uri="/WEB-INF/tlds/mytags.tld" %>

<%-- ── Tag file library (directory in WEB-INF/tags/) ─────── --%>
<%@ taglib prefix="ui"  tagdir="/WEB-INF/tags/ui" %>

<%-- After declaration, use as: <prefix:tagname attr="..."> --%>
<c:forEach items="${products}" var="p">
    <p>${fn:escapeXml(p.name)}</p>
</c:forEach>
<fmt:formatNumber value="${price}" type="currency" />
```

CODE NOTES

→ `uri="jakarta.tags.core"` — short-form URI used in JSTL 3.x / Jakarta EE 10+

→ `uri="http://java.sun.com/..."` — legacy full URI; still works if using javax JSTL JARs

→ `tagdir="/WEB-INF/tags/..."` — use instead of `uri` for tag file directories; mutually exclusive

→ `prefix` — any valid XML name; convention is the standard short form (`c` , `fmt` , `fn`)

→ TLD auto-discovery — container scans `WEB-INF/lib/*.jar` META-INF for `.tld` files on startup

MODERN EQUIVALENT

- Thymeleaf dialects replace taglibs (`xmlns:th`)

PREFER / AVOID

✗ ~~`java.sun.com` JSTL URIs on Jakarta EE 10~~ → ✓ `jakarta.tags.core`

✗ ~~unused taglibs~~ → ✓ **only the prefixes you use**

JSTL LIBRARY URIS — JAKARTA VS JAVAX

LIBRARY	JAKARTA EE 9+ URI	LEGACY JAVAX URI
Core (c:)	jakarta.tags.core	…/jstl/core
Formatting (fmt:)	jakarta.tags.fmt	…/jstl/fmt
Functions (fn:)	jakarta.tags.functions	…/jstl/functions
SQL (sql:)	jakarta.tags.sql	…/jstl/sql
XML (x:)	jakarta.tags.xml	…/jstl/xml

TLD RESOLUTION ORDER

LOCATION	HOW RESOLVED
WEB-INF/lib/*.jar	Container scans `META-INF/*.tld` inside each JAR
WEB-INF/*.tld	Direct file path in `uri` attribute
WEB-INF/tags/	Tag files; use `tagdir` not `uri`
Implicit (container)	jsp: prefix always available — no taglib needed

⚠ COMMON MISTAKES

- Using Jakarta short URIs (`jakarta.tags.core`) with the old `javax.servlet.jsp.jstl` JAR — the URIs must match what the JAR's TLD declares
- Omitting the taglib directive but still using `<c:forEach>` — the tag is treated as unknown XML and either errors or renders literally
- Using `tagdir` and `uri` together on the same directive — they are mutually exclusive; the container rejects the page
- Placing the JSTL JAR in `tomcat/lib/` instead of `WEB-INF/lib/` — it may work in one container version but breaks portability

✓ TIPS

- Declare all taglibs at the top of each JSP before any output — directives after template text can cause whitespace issues in some containers
- Extract repeated taglib declarations into a shared fragment (`taglibs.jspf`) and include it with `<%@ include %>` in every page
- Match your JSTL JAR version to your Jakarta EE version — JSTL 3.x for Jakarta EE 9+, JSTL 1.2 for older javax stacks
- Use `tagdir` (not `uri`) for tag file libraries in `WEB-INF/tags/` — no TLD file required for tag files

QUICK SUMMARY

- `<%@ taglib prefix="c" uri="jakarta.tags.core" %>` — import JSTL core
- `uri` — logical name; resolved against TLDs in JARs/WEB-INF
- `tagdir` — use for `WEB-INF/tags/` tag file libraries instead of `uri`
- Jakarta EE 9+ — short URIs (`jakarta.tags.*`); legacy uses full URLs
- TLD discovery — automatic from `META-INF/*.tld` in `WEB-INF/lib/` JARs

Sheet 07

Implicit Objects — request & response

getParameter · getHeader · getMethod · setContentType · sendRedirect · HttpServletRequest

REQUEST & RESPONSE IMPLICIT OBJECTS

Every JSP has nine **implicit objects** available without declaration — they are local variables in the generated `_jspService()` method. `request` is the `HttpServletRequest` for the current call; `response` is the `HttpServletResponse`. In practice, most view JSPs read from `request` and rarely touch `response` directly (the container handles headers). Output goes through the `out` implicit object, not `response.getWriter()`.

KEY FACTS

- `request` type: `jakarta.servlet.http.HttpServletRequest`
- `response` type: `jakarta.servlet.http.HttpServletResponse`
- Do not call `response.getWriter()` in a JSP — it conflicts with the container-managed `JspWriter out`
- `getParameter()` returns `null` if the param is absent — always null-check or use a default
- Calling `response.sendRedirect()` after output has begun throws an `IllegalStateException`

METHOD NOTES

- → `getParameter(name)` — first value of named param; null if missing
- → `getParameterValues(name)` — all values of a multi-value param (checkboxes)
- → `getParameterMap()` — all params as `Map<String,String[]>`
- → `getAttribute(name)` — reads request-scope attribute set by a servlet
- → `getContextPath()` — prefix for all app URLs; use in hrefs
- → `sendRedirect(url)` — 302 response; must be before output; always `return` after
- → `setHeader(name, value)` — set response header; must be before output

PREFER / AVOID

✗ ~~`request.getParameter`~~ ~~in the view~~ → **✓ controller reads params, passes a model**

SECURITY CHECK

- Every parameter and header is attacker-controlled

COMMON REQUEST & RESPONSE USAGE

```
<%-- ── Reading request parameters ──────────────────────── --%>
<%
    String name  = request.getParameter("name");       // null if absent
    String[] ids = request.getParameterValues("id");   // multi-value
    String q     = request.getParameter("q");
    if (q == null) q = "";
%>

<%-- ── Request metadata ─────────────────────────────────── --%>
<%= request.getMethod() %>            <%-- GET / POST --%>
<%= request.getRequestURI() %>        <%-- /myapp/home --%>
<%= request.getContextPath() %>       <%-- /myapp --%>
<%= request.getServletPath() %>       <%-- /home --%>
<%= request.getRemoteAddr() %>        <%-- client IP --%>

<%-- ── Reading headers ──────────────────────────────────── --%>
<%= request.getHeader("User-Agent") %>
<%= request.getHeader("Accept-Language") %>

<%-- ── Redirect (must be before any output) ─────────────── --%>
<%
    if (session.getAttribute("user") == null) {
        response.sendRedirect(request.getContextPath() + "/login");
        return;   // stop further processing
    }
%>

<%-- ── Setting response headers ─────────────────────────── --%>
<% response.setHeader("Cache-Control", "no-store"); %>
```

MODERN EQUIVALENT

- Spring: `@RequestParam`, `HttpServletResponse` in the controller
- Next.js: `searchParams`, `redirect()`

HTTPSERVLETREQUEST — KEY METHODS

METHOD	RETURNS
`getParameter(name)`	String (null if absent)
`getParameterValues(name)`	String[]
`getParameterMap()`	Map<String,String[]>
`getAttribute(name)`	Object (request scope)
`setAttribute(name, val)`	void
`getHeader(name)`	String
`getMethod()`	"GET" / "POST" / …
`getRequestURI()`	String — full path
`getContextPath()`	String — app root prefix
`getServletPath()`	String — path after context
`getRemoteAddr()`	String — client IP
`getSession()`	HttpSession

HTTPSERVLETRESPONSE — KEY METHODS

METHOD	NOTES
`sendRedirect(url)`	302; call before output; then return
`setStatus(code)`	Set HTTP status (200, 404, …)
`setContentType(type)`	Override page directive at runtime
`setHeader(name, val)`	Set arbitrary response header
`addCookie(cookie)`	Add a Set-Cookie header
`encodeURL(url)`	Append session ID if cookies off
`encodeRedirectURL(url)`	Same, for redirect URLs

⚠ COMMON MISTAKES

- Calling `response.getWriter()` in a JSP — the container already opened `JspWriter out`; a second writer throws `IllegalStateException`
- Not calling `return` after `sendRedirect()` — the rest of the JSP continues executing, producing output after headers are committed
- Building redirect URLs without `request.getContextPath()` — hardcoded paths break when the app is deployed under a different context
- Displaying `getParameter()` output directly without escaping — user-controlled input rendered raw is an XSS vulnerability

✓ TIPS

- Always use `${pageContext.request.contextPath}` (EL) or `request.getContextPath()` in href/src attributes — never hardcode the app prefix
- Use `response.encodeURL(url)` on every internal link to support session tracking when cookies are disabled
- Read parameters in the servlet, store results with `request.setAttribute()`, then forward to JSP — keeps JSP free of parameter-reading logic
- Use `<c:out value="${param.q}" />` instead of `<%= request.getParameter("q") %>` — auto-escapes HTML entities

QUICK SUMMARY

- `request` — HttpServletRequest; read params, headers, attributes
- `response` — HttpServletResponse; redirect, set headers
- `getParameter(name)` — returns null if absent; always check
- `sendRedirect(url)` — must precede output; always `return` after
- `getContextPath()` — use in all absolute links inside JSPs

Sheet 08

Implicit Objects — session & application

HttpSession · setAttribute · invalidate · getServletContext · session timeout · context-wide state

SESSION & APPLICATION

`session` is the `HttpSession` for the current user — it persists across multiple requests from the same browser. `application` is the `ServletContext` — it is shared across *all* users and all JSPs in the entire web application for its lifetime. Use session for per-user state (login, cart) and application for app-wide configuration or shared caches.

KEY FACTS

- `session` type: `jakarta.servlet.http.HttpSession`
- `application` type: `jakarta.servlet.ServletContext`
- `session` is unavailable when `<%@ page session="false" %>` is set
- `application` attributes are shared across all threads — always synchronize writes
- Session timeout defaults to 30 minutes; configurable in `web.xml` via `<session-timeout>`
- Session is tracked via cookie (`JSESSIONID`) or URL rewriting — use `response.encodeURL()` for URL rewriting support

METHOD NOTES

- → `session.setAttribute(k,v)` — stores object under key for this user
- → `session.getAttribute(k)` — retrieves; returns null if absent
- → `session.removeAttribute(k)` — deletes one attribute
- → `session.invalidate()` — destroys session; all attributes lost
- → `session.setMaxInactiveInterval(sec)` — override timeout for this session
- → `application.getInitParameter(k)` — reads `<context-param>` from web.xml
- → `application.getRealPath(path)` — converts web path to filesystem path

PREFER / AVOID

- ✗ ~~large objects in the session~~ → ✓ **IDs only; load on demand**
- ✗ ~~`session.setAttribute` from views~~ → ✓ **controllers / services**

SECURITY CHECK

- New session id on login (fixation): `request.changeSessionId()`
- `invalidate()` on logout; `HttpOnly` + `Secure` cookies

SESSION & APPLICATION USAGE

```
<%-- ── Session: store and retrieve user data ──────────────── --%>
<%
    // Write (typically done in the servlet, not the JSP)
    session.setAttribute("user", userObject);
    session.setAttribute("cartSize", 3);
%>

<%-- Read in JSP via EL (preferred) --%>
<p>Welcome, ${sessionScope.user.name}</p>
<p>Cart: ${sessionScope.cartSize} items</p>

<%-- Read via scriptlet --%>
<% User u = (User) session.getAttribute("user"); %>

<%-- Session metadata --%>
<%= session.getId() %>              <%-- JSESSIONID value --%>
<%= session.isNew() %>              <%-- true on first request --%>
<%= session.getMaxInactiveInterval() %> <%-- timeout in seconds --%>

<%-- Invalidate on logout --%>
<%
    session.invalidate();
    response.sendRedirect(request.getContextPath() + "/login");
    return;
%>

<%-- ── Application: app-wide shared state ─────────────────── --%>
<%
    // Read a context init param (from web.xml <context-param>)
    String apiUrl = application.getInitParameter("api.base.url");

    // Store/read app-wide attribute
    application.setAttribute("appVersion", "2.1.0");
%>
<p>API: ${applicationScope.appVersion}</p>
<p>Real path: <%= application.getRealPath("/WEB-INF/config.xml") %></p>
```

MODERN EQUIVALENT

- Spring Session / `@SessionAttributes` · stateless JWT for APIs

HTTPSESSION — KEY METHODS

METHOD	DESCRIPTION
`getAttribute(name)`	Get stored object; null if absent
`setAttribute(name, val)`	Store object in session
`removeAttribute(name)`	Delete one attribute
`invalidate()`	Destroy session entirely
`getId()`	JSESSIONID token string
`isNew()`	True if created this request
`getCreationTime()`	Epoch ms when created
`getLastAccessedTime()`	Epoch ms of last access
`getMaxInactiveInterval()`	Idle timeout in seconds
`setMaxInactiveInterval(s)`	Override idle timeout

SERVLETCONTEXT (APPLICATION) — KEY METHODS

METHOD	DESCRIPTION
`getAttribute(name)`	Get app-wide attribute
`setAttribute(name, val)`	Store app-wide attribute
`removeAttribute(name)`	Delete app-wide attribute
`getInitParameter(name)`	Read <context-param> from web.xml
`getRealPath(path)`	Web path → filesystem path
`getResourceAsStream(path)`	Open file relative to context root
`log(message)`	Write to container log
`getContextPath()`	App context prefix (e.g. /myapp)

⚠ COMMON MISTAKES

- Accessing `session` on a page with `<%@ page session="false" %>` — the variable doesn't exist, causing a compile error
- Writing to `application` attributes from multiple threads without synchronization — `ServletContext` is not thread-safe for concurrent writes
- Not calling `session.invalidate()` on logout — the old session (and its attributes) persist until timeout, a security risk
- Storing non-serializable objects in session — causes `NotSerializableException` if the container persists sessions to disk for failover

✓ TIPS

- Access session data in JSP via EL `${sessionScope.key}` — cleaner than a scriptlet cast and null-safe
- Set `<%@ page session="false" %>` on all pages that don't need it — avoids unnecessary session creation and improves scalability
- Use `application.getInitParameter()` for environment-specific config (base URLs, feature flags) — change `web.xml` without touching JSP code
- Keep session attributes minimal and serializable — large session payloads hurt clustering and memory

QUICK SUMMARY

- `session` — per-user state; HttpSession; persists across requests
- `application` — app-wide state; ServletContext; shared by all users
- `session.invalidate()` — destroys session; call on logout
- `application.getInitParameter()` — reads context-param from web.xml
- `${sessionScope.key}` — EL access to session attributes

Implicit Objects — out, config, pageContext, page, exception

THE REMAINING FIVE IMPLICIT OBJECTS

Beyond `request`, `response`, `session`, and `application`, JSP provides five more implicit objects. `out` is the buffered writer that sends text to the response. `pageContext` is a gateway to all other implicit objects and all four scopes — useful in tag handlers. `config` gives access to servlet init parameters. `page` is `this` — rarely needed. `exception` holds the unhandled `Throwable` on error pages only.

KEY FACTS

- `out` type: `jakarta.servlet.jsp.JspWriter` — a buffered `PrintWriter` equivalent
- `pageContext` type: `jakarta.servlet.jsp.PageContext` — gateway to everything
- `config` type: `jakarta.servlet.ServletConfig` — reads `<init-param>` in web.xml
- `page` type: `Object` (actually `this`) — essentially never useful in a JSP
- `exception` is only available when `isErrorPage="true"`; null otherwise
- Never mix `out` and `response.getWriter()` — they buffer separately and produce garbled output

USAGE EXAMPLES

```
<%-- ── out: write to response ──────────────────────────── --%>
<% out.print("Hello"); %>              <%-- same as <%= "Hello" %> --%>
<% out.println("<br>"); %>             <%-- newline after content --%>
<% out.flush(); %>                     <%-- flush buffer manually --%>
<%= out.getBufferSize() %>             <%-- e.g. 8192 (bytes) --%>
<%= out.getRemaining() %>              <%-- bytes left in buffer --%>

<%-- ── pageContext: scope-crossing access ──────────────── --%>
<%
    // Get any implicit object via pageContext
    HttpServletRequest  req  = (HttpServletRequest)  pageContext.getRequest();
    HttpServletResponse res  = (HttpServletResponse) pageContext.getResponse();
    HttpSession         ses  = pageContext.getSession();
    ServletContext      ctx  = pageContext.getServletContext();

    // Search all scopes for an attribute
    Object val = pageContext.findAttribute("username");
    // → checks page → request → session → application in order

    // Scope-specific set/get
    pageContext.setAttribute("msg", "Hi", PageContext.REQUEST_SCOPE);
%>

<%-- ── config: servlet init parameters ─────────────────── --%>
<%= config.getInitParameter("adminEmail") %>
<%= config.getServletName() %>

<%-- ── exception: available on error pages only ─────────── --%>
<%-- In error.jsp: <%@ page isErrorPage="true" %> --%>
<p>Error: <%= exception.getMessage() %></p>
<% exception.printStackTrace(new java.io.PrintWriter(out)); %>
```

METHOD NOTES

- → `out.print(val)` — writes to response buffer; no newline
- → `out.flush()` — sends buffer to client; after this, headers are committed
- → `pageContext.findAttribute(name)` — searches all four scopes in order: page → request → session → application
- → `pageContext.setAttribute(k,v,scope)` — scope constants: PAGE_SCOPE, REQUEST_SCOPE, SESSION_SCOPE, APPLICATION_SCOPE
- → `pageContext.forward(path)` — equivalent to RequestDispatcher.forward()
- → `config.getInitParameter(name)` — reads JSP servlet's own `<init-param>` from web.xml
- → `exception` — the Throwable that caused the error; only on `isErrorPage="true"` pages

ALL NINE IMPLICIT OBJECTS

OBJECT	TYPE	SCOPE
`request`	HttpServletRequest	Request
`response`	HttpServletResponse	Page
`session`	HttpSession	Session
`application`	ServletContext	Application
`out`	JspWriter	Page
`pageContext`	PageContext	Page
`config`	ServletConfig	Page
`page`	Object (this)	Page
`exception`	Throwable	Page (error pages only)

PAGECONTEXT SCOPE CONSTANTS

CONSTANT	VALUE	SCOPE
`PAGE_SCOPE`	1	Current page only
`REQUEST_SCOPE`	2	Current request
`SESSION_SCOPE`	3	User's session
`APPLICATION_SCOPE`	4	Entire application

JSPWRITER METHOD	NOTES
`print(val)`	Write without newline
`println(val)`	Write + newline char
`flush()`	Commit buffer to client
`getBufferSize()`	Total buffer size (bytes)
`getRemaining()`	Remaining buffer space

MODERN EQUIVALENT

- Thymeleaf: `${#request}`, `${#session}` utility objects (avoid in views)

PREFER / AVOID

✗ ~~`out.print()` in pages~~ → ✓ **template text + EL**

✗ ~~`pageContext` gymnastics~~ → ✓ **model attributes**

⚠ COMMON MISTAKES

- Accessing `exception` without `isErrorPage="true"` — the variable is not declared; causes a compile error
- Calling `out.flush()` before all headers are set — once the buffer is flushed, response headers are committed and cannot be changed
- Using `page` (i.e. `this`) — it is typed as `Object`, so it requires a cast to be useful; `pageContext` provides the same access more cleanly
- Reading `config.getInitParameter()` expecting application-wide params — `config` reads the JSP servlet's own `<init-param>`; use `application.getInitParameter()` for context params

✓ TIPS

- Use `pageContext.findAttribute("name")` in custom tag handlers to search all scopes without knowing which scope holds the value
- On error pages, log `exception.getMessage()` via `application.log()` rather than printing stack traces to the response
- Prefer EL (`${}`) over `out.print()` for all output — EL is cleaner and does not require a scriptlet block
- Use `pageContext.getELContext()` when writing tag handlers that need to evaluate EL expressions programmatically

QUICK SUMMARY

- `out` — JspWriter; buffered response writer; prefer EL over out.print()
- `pageContext` — gateway to all implicit objects and scopes
- `pageContext.findAttribute(name)` — searches page→request→session→app
- `config` — servlet init-params; rarely used in view JSPs
- `exception` — Throwable on error pages; requires `isErrorPage="true"`

Page & Request Scope

THE TWO SHORTEST-LIVED SCOPES

JSP defines four attribute scopes, from shortest to longest lived. **Page scope** exists only for the duration of processing a single JSP page — attributes are gone the moment the response is dispatched. **Request scope** survives across a `RequestDispatcher.forward()` call, making it the primary mechanism for a servlet to pass data to a JSP view. Both scopes are accessed through their respective objects: `pageContext` for page scope, `request` for request scope.

KEY FACTS

- Page scope attributes are NOT shared with included files via `<jsp:include>` — each dispatched resource gets its own page scope
- Request scope IS shared across a `forward()` — it is the standard way servlets hand data to JSPs
- Request scope is also shared with static includes (`<%@ include %>`) because they become one servlet
- EL accesses request attributes via `${requestScope.key}` or just `${key}` (EL searches all scopes)
- `getAttribute()` returns `Object` — cast to the expected type in scriptlets; EL handles this automatically

SCOPE USAGE PATTERNS

```
// ── In the Servlet (controller) ──────────────────────────
List<Product> products = productService.findAll();
request.setAttribute("products", products);     // request scope
request.setAttribute("pageTitle", "Catalogue"); // request scope

RequestDispatcher rd = request.getRequestDispatcher(
        "/WEB-INF/jsp/catalogue.jsp");
rd.forward(request, response);  // attributes travel with request
```

```
<%-- ── In the JSP (view) — EL access (preferred) ──────────── --%>
<title>${requestScope.pageTitle}</title>
<title>${pageTitle}</title>          <%-- shorthand — same result --%>

<c:forEach items="${products}" var="p">
    <p>${p.name} — ${p.price}</p>
</c:forEach>

<%-- ── Scriptlet access (cast required) ──────────────────── --%>
<%
    List<Product> prods =
        (List<Product>) request.getAttribute("products");
%>

<%-- ── Page scope: set and read within same JSP only ──────── --%>
<%
    pageContext.setAttribute("localMsg", "computed value");
%>
<p>${pageScope.localMsg}</p>

<%-- ── Remove an attribute ──────────────────────────────────── --%>
<% request.removeAttribute("products"); %>
```

CODE NOTES

- → `request.setAttribute("key", val)` — stores in request scope; survives forward()
- → `request.getAttribute("key")` — retrieves; returns null if absent
- → `rd.forward(request, response)` — passes the same request (and its attributes) to the JSP
- → `${products}` — EL shorthand; searches all scopes in order
- → `${requestScope.products}` — explicit scope; unambiguous; use when shadowing is possible
- → `pageContext.setAttribute("key", val)` — page scope only; not shared with any included resource

MODERN EQUIVALENT

- Spring `Model` attributes = request scope

PREFER / AVOID

- ✗ ~~page scope for data from servlets~~ → ✓ **request attributes**
- ✗ ~~`getAttribute` + casts~~ → ✓ `${name}` **in EL**

SCOPE LIFETIME & VISIBILITY

SCOPE	LIVES UNTIL	SHARED ACROSS
`page`	End of JSP processing	Nothing — page only
`request`	Response committed	forward(), static include
`session`	Session timeout / invalidate	All requests same user
`application`	Server shutdown / undeploy	All users, all pages

EL SCOPE ACCESS & SEARCH ORDER

EL SYNTAX	SCOPE TARGETED
`${pageScope.key}`	Page only
`${requestScope.key}`	Request only
`${sessionScope.key}`	Session only
`${applicationScope.key}`	Application only
`${key}`	Searches: page → request → session → application

⚠ COMMON MISTAKES

- Setting a request attribute in the JSP after a `<jsp:include>` and expecting the included page to see it — `<jsp:include>` runs its own page scope; use the request scope and set before including
- Using bare `${key}` when an attribute of the same name exists in multiple scopes — the most recently set one in the search order wins; use explicit `${requestScope.key}` to be unambiguous
- Expecting page-scope attributes to survive a redirect — a redirect is a new request; page and request scope are both gone
- Forgetting to cast `request.getAttribute()` in scriptlets — it returns `Object`; without a cast, calling methods on it is a compile error

✓ TIPS

- Always set request attributes in the *servlet*, not the JSP — the servlet is the controller; the JSP only reads them
- Use explicit scope prefixes in EL (`${requestScope.user}`) in pages that have a lot of attributes — prevents accidental scope shadowing
- Prefer `request.setAttribute()` over URL query parameters for passing data — avoids length limits and keeps data out of the browser address bar
- For data that must survive a redirect (Post/Redirect/Get pattern), store in session briefly and remove after reading in the next request

QUICK SUMMARY

- `page scope` — lives for one JSP page only; use pageContext to set
- `request scope` — survives forward(); servlet→JSP data channel
- `request.setAttribute("k", v)` — primary way servlet passes data to view
- `${requestScope.key}` — explicit EL; avoids scope ambiguity
- Redirect kills request scope — use session for Post/Redirect/Get

Sheet 11

Session & Application Scope

sessionScope · applicationScope · cross-request state · invalidate · concurrent access · context attributes

LONGER-LIVED SCOPES

Session scope persists across multiple requests from the same browser until the session times out or is invalidated — it is the right place for per-user state like login identity, shopping carts, and preferences. **Application scope** persists for the entire lifetime of the web application and is shared by every user simultaneously — use it only for truly global, read-mostly data like configuration values, lookup tables, or shared caches.

KEY FACTS

- Session scope is per-user; application scope is per-application (all users share it)
- Application-scope writes from multiple threads require synchronization — the container does not lock for you
- Objects stored in session must be `Serializable` if the container persists or replicates sessions
- EL accesses via `${sessionScope.key}` and `${applicationScope.key}`
- Session is identified by a `JSESSIONID` cookie (or URL param); never expose this value in pages

CODE NOTES

- → `session.setAttribute(k,v)` — stores for this user across requests
- → `session.invalidate()` — destroys all session data; use on logout
- → `setMaxInactiveInterval(sec)` — override idle timeout per session
- → `application.getAttribute(k)` — shared across ALL users; read is safe
- → `application.getInitParameter(k)` — reads `<context-param>` from web.xml; read-only after startup
- → Write to application scope only from `ServletContextListener.contextInitialized()` to avoid threading issues

PREFER / AVOID

- ✗ ~~mutable maps in application scope~~ → ✓ **thread-safe services**
- ✗ ~~session for cross-request caches~~ → ✓ **a cache (Caffeine / Redis)**

SECURITY CHECK

- Application scope is shared by all users
- Never store user data there

SESSION & APPLICATION SCOPE PATTERNS

```
<%-- ── Session scope: per-user state ──────────────────────── --%>

<%-- Write (done in servlet after successful login) --%>
<%
    session.setAttribute("currentUser", user);
    session.setAttribute("role", "ADMIN");
    session.setMaxInactiveInterval(1800); // 30 min
%>

<%-- Read in JSP --%>
<p>Logged in as: ${sessionScope.currentUser.fullName}</p>
<c:if test="${sessionScope.role == 'ADMIN'}">
    <a href="admin">Admin Panel</a>
</c:if>

<%-- Logout: destroy session --%>
<%
    session.invalidate();
    response.sendRedirect(request.getContextPath() + "/login");
    return;
%>

<%-- ── Application scope: app-wide shared data ─────────────── --%>

<%-- Write once at startup (in a ServletContextListener, not JSP) --%>
<%
    // Reading is fine in JSP; writing should be done in a listener
    String version = (String) application.getAttribute("appVersion");
    Integer visits  = (Integer) application.getAttribute("visitCount");
%>

<%-- EL read --%>
<footer>Version: ${applicationScope.appVersion}</footer>

<%-- Read init param from web.xml <context-param> --%>
<%= application.getInitParameter("supportEmail") %>
```

MODERN EQUIVALENT

- Spring beans (singleton) instead of application scope

SESSION VS APPLICATION SCOPE

FEATURE	SESSION	APPLICATION
Shared by	One user	All users
Lifetime	Until timeout/invalidate	Until undeploy/shutdown
Object via	session implicit	application implicit
EL prefix	sessionScope	applicationScope
Thread safety	Per-user; mostly safe	Shared; sync writes
Serializable?	Required for clustering	Not required
Best for	Login state, cart, prefs	Config, lookup tables

HTTPSESSION API QUICK REFERENCE

METHOD	DESCRIPTION
`setAttribute(k, v)`	Store object
`getAttribute(k)`	Retrieve; null if absent
`removeAttribute(k)`	Delete one key
`invalidate()`	Destroy entire session
`getId()`	JSESSIONID value
`isNew()`	Created this request?
`getMaxInactiveInterval()`	Timeout in seconds
`setMaxInactiveInterval(s)`	Override idle timeout
`getAttributeNames()`	Enumeration of all keys

⚠ COMMON MISTAKES

- Writing mutable counters to `application` scope from JSP scriptlets — concurrent requests produce race conditions; use a `ServletContextListener` or synchronized block
- Storing large objects (full result sets, byte arrays) in session — each user's session holds this in memory; it degrades performance under load
- Not invalidating the session on logout — the old session token remains valid until timeout, leaving the account accessible
- Storing non-serializable objects in session — works until the container tries to persist or replicate sessions, then throws `NotSerializableException`

✓ TIPS

- Initialize application-scope data in a `ServletContextListener` (`contextInitialized`) — runs once at startup, thread-safe, no JSP scriptlet required
- After login, call `request.getSession(false)` and then `invalidate()` and `request.getSession(true)` to rotate the session ID and prevent session fixation
- Keep session payloads small — store only a user ID in session and re-fetch user data from the database per-request
- Set `<session-config><session-timeout>30</session-timeout></session-config>` in web.xml rather than hardcoding it in JSPs

QUICK SUMMARY

- `session scope` — per-user; persists across requests; use for login state
- `application scope` — all users; app lifetime; use for config/caches
- `session.invalidate()` — required on logout for security
- `${sessionScope.key}` / `${applicationScope.key}` — EL access
- Application scope writes — synchronize or use a listener

PageContext API & Scope Resolution

PAGECONTEXT AS A UNIFIED API

`PageContext` is the single object that provides programmatic access to all four JSP scopes and all nine implicit objects from one place. It is most useful inside **custom tag handlers**, which do not have direct access to implicit objects. In JSPs, `pageContext` is available as an implicit object. Its key value is `findAttribute(name)`, which searches all four scopes in order — the same search EL performs when you write `${name}`.

KEY FACTS

- Scope search order for `findAttribute()`: page → request → session → application
- This is identical to the order EL uses when you write `${name}` without a scope prefix
- Tag handlers receive `pageContext` via `setPageContext()` — it is their only gateway to the web context
- `getELContext()` returns the `ELContext` needed to evaluate EL expressions programmatically
- Scope integer constants are on `PageContext` itself: `PageContext.REQUEST_SCOPE` etc.

PAGECONTEXT API USAGE

```
// ── In a tag handler (no implicit objects available) ──────
public class MyTag extends SimpleTagSupport {
    public void doTag() throws JspException, IOException {
        PageContext pc = (PageContext) getJspContext();

        // Access any implicit object
        HttpServletRequest  req = (HttpServletRequest)  pc.getRequest();
        HttpServletResponse res = (HttpServletResponse) pc.getResponse();
        HttpSession         ses = pc.getSession();
        ServletContext      ctx = pc.getServletContext();
        JspWriter           out = pc.getOut();

        // Search all scopes (page→request→session→application)
        Object user = pc.findAttribute("currentUser");

        // Scope-targeted get/set
        pc.setAttribute("msg", "Hello", PageContext.REQUEST_SCOPE);
        Object val = pc.getAttribute("msg", PageContext.REQUEST_SCOPE);
        pc.removeAttribute("msg", PageContext.REQUEST_SCOPE);

        // Forward from inside a tag
        pc.forward("/WEB-INF/jsp/result.jsp");

        // Include another resource
        pc.include("/WEB-INF/jsp/widget.jsp");

        // Write output
        out.print("<p>Tag output</p>");
    }
}
```

METHOD NOTES

- → `findAttribute(name)` — searches page→request→session→app; returns first match or null
- → `getAttribute(name, scope)` — single-scope targeted get
- → `setAttribute(name, val, scope)` — store in a specific scope
- → `removeAttribute(name, scope)` — delete from specific scope
- → `getRequest()` — returns `ServletRequest`; cast to `HttpServletRequest`
- → `getOut()` — returns `JspWriter` for writing response output
- → `getELContext()` — needed to programmatically evaluate EL in tag handlers
- → `forward(path)` / `include(path)` — dispatch to another resource

MODERN EQUIVALENT

- Spring resolves model → view variables for you

PREFER / AVOID

- ✗ ~~`findAttribute` everywhere~~ → ✓ **explicit** `requestScope.x`
- ✗ ~~same name in two scopes~~ → ✓ **unique attribute names**

SCOPE INTEGER CONSTANTS

CONSTANT	INT	OBJECT ACCESSED VIA
`PAGE_SCOPE`	1	pageContext / JspContext
`REQUEST_SCOPE`	2	request
`SESSION_SCOPE`	3	session
`APPLICATION_SCOPE`	4	application (ServletContext)

PAGECONTEXT IMPLICIT OBJECT GETTERS

METHOD	RETURNS
`getRequest()`	ServletRequest (cast to Http…)
`getResponse()`	ServletResponse
`getSession()`	HttpSession
`getServletContext()`	ServletContext
`getOut()`	JspWriter
`getServletConfig()`	ServletConfig
`getELContext()`	ELContext
`getException()`	Exception (error pages)

FINDATTRIBUTE VS GETATTRIBUTE

METHOD	SCOPE SEARCHED
`findAttribute(name)`	All 4 scopes in order
`getAttribute(name)`	Page scope only
`getAttribute(name, scope)`	Specified scope only
`${name}`	All 4 scopes (EL equivalent)
`${requestScope.name}`	Request scope only (EL)

⚠ COMMON MISTAKES

- Calling `pageContext.getAttribute("key")` (no scope argument) and expecting it to search all scopes — the no-arg version only looks in page scope; use `findAttribute()` for a full search
- Casting `pageContext.getJspContext()` to `PageContext` in a `SimpleTag` without checking — `getJspContext()` returns `JspContext`; the cast is valid in a servlet container but not in all EL environments
- Using `pageContext.forward()` in a JSP scriptlet after output has started — once the buffer is flushed, forward throws `IllegalStateException`

✓ TIPS

- In tag handlers, always use `pageContext.findAttribute(name)` to mirror what EL does — users of your tag will expect `${name}` resolution semantics
- Prefer explicit-scope variants (`getAttribute(name, REQUEST_SCOPE)`) over `findAttribute()` when you know which scope you need — it's faster and unambiguous
- Use `pageContext.getELContext()` with `ExpressionFactory` to evaluate user-supplied EL attribute values in custom tags
- Access `pageContext` directly in JSP scriptlets when you need to programmatically forward or set attributes in multiple scopes in one block

QUICK SUMMARY

- `pageContext` — gateway to all implicit objects and all scopes
- `findAttribute(name)` — searches page→request→session→app (same as EL ${name})
- `getAttribute(name)` — page scope only; use scoped variant for others
- `setAttribute(name, val, scope)` — store in a specific scope
- Tag handlers — use `pageContext` as the only gateway to the web context

Sheet 13

EL Syntax & Value Expressions

${} immediate · #{} deferred · dot operator · bracket operator · null-safe · isELIgnored

EXPRESSION LANGUAGE FUNDAMENTALS

The **Expression Language (EL)** is JSP's primary syntax for reading data and calling functions without scriptlets. `${expr}` is an *immediate* value expression — it evaluates and outputs when the page renders. `#{expr}` is a *deferred* expression — it is evaluated later by a component framework (e.g. JSF). EL is null-safe: accessing a property of a null object returns empty string, not a `NullPointerException`.

KEY FACTS

- EL is enabled by default in JSP 2.0+; disable per-page with `<%@ page isELIgnored="true" %>`
- Null safety: `${null.anything}` → empty string, not NPE
- Dot operator `${bean.name}` calls `getName()` via JavaBean convention
- Bracket operator `${map["key"]}` works for maps, lists, arrays, and bean properties
- EL does not output HTML — use `<c:out>` or `fn:escapeXml()` to escape before printing user data
- To output a literal `${`, escape it: `\${`

EL EXPRESSION PATTERNS

```
<%-- ── Immediate vs deferred --%>
${user.name}          <%-- immediate: evaluates now (JSP/JSTL) --%>
#{user.name}          <%-- deferred: evaluated by framework (JSF) --%>

<%-- ── Dot vs bracket operator --%>
${user.firstName}     <%-- calls user.getFirstName() --%>
${user["firstName"]}  <%-- identical to dot for beans --%>
${map["key-name"]}    <%-- bracket required: key has hyphen --%>
${list[0]}            <%-- array/list index access --%>
${map[varName]}       <%-- dynamic key from another variable --%>

<%-- ── Null safety --%>
${user.address.city}  <%-- "" if user, address, or city is null --%>

<%-- ── Composite expressions --%>
<a href="${pageContext.request.contextPath}/orders/${order.id}">
    View Order #${order.id}
</a>
<img src="${pageContext.request.contextPath}/img/${product.image}"
     alt="${product.name}">

<%-- ── Escape literal ${ --%>
\${not evaluated}     <%-- renders as: ${not evaluated} --%>

<%-- ── EL in tag attributes --%>
<c:if test="${user.role == 'ADMIN'}">...</c:if>
<c:forEach items="${products}" var="p">...</c:forEach>

<%-- ── isELIgnored: treat ${ as literal --%>
<%@ page isELIgnored="true" %>
${this is printed literally}
```

SYNTAX NOTES

- → `${expr}` — immediate; renders value to response when page processes
- → `#{expr}` — deferred; for JSF/component frameworks; not evaluated by JSTL
- → `bean.property` — calls getter: `getProperty()` or `isProperty()` for booleans
- → `map["key"]` — use brackets when key has special chars or is dynamic
- → `list[0]` — zero-based index for arrays and `List` types
- → Null mid-chain — any null in a chain returns empty string; no exception
- → `\${` — literal dollar-brace; EL does not evaluate escaped expressions

MODERN EQUIVALENT

- Thymeleaf `${…}` (Spring EL) escapes with `th:text`
- JSX escapes by default

PREFER / AVOID

- ✗ ~~scriptlet expressions~~ → ✓ `${bean.prop}`
- ✗ ~~`${x}` straight into HTML~~ → ✓ `<c:out value="${x}"/>`

SECURITY CHECK

- `${…}` in template text is NOT escaped
- Wrap untrusted values in `<c:out>` or `fn:escapeXml`

DOT VS BRACKET — WHEN TO USE EACH

SITUATION	USE	EXAMPLE
Bean property	Either	${user.name}
Map with simple key	Either	${map.key}
Map with special-char key	Bracket	${map["my-key"]}
List / array index	Bracket	${list[2]}
Dynamic key variable	Bracket	${map[keyVar]}
Nested map	Both	${a.b["c"]}

EL TYPE COERCION BASICS

VALUE	COERCES TO	RESULT
null	String	""
null	Number	0
null	Boolean	false
"42"	int/long	42
"true"	boolean	true
42	String	"42"

⚠ COMMON MISTAKES

- Outputting `${param.userInput}` directly — EL does not HTML-escape; a value like `<script>` renders as live HTML; always use `<c:out value="${...}" />` for user data
- Using dot notation for a map key containing a hyphen (`${map.my-key}`) — EL parses this as subtraction; use `${map["my-key"]}`
- Forgetting that `${user.name}` calls `getName()` — if the bean method is named `fetchName()` instead of `getName()`, EL returns null silently
- Using `#{}` in a plain JSP/JSTL page — deferred expressions are not evaluated by the JSP engine; they render as empty string or cause a parse error

✓ TIPS

- Use `${pageContext.request.contextPath}` in all link and resource `href` / `src` attributes to build portable context-relative URLs
- Chain null-safe access freely: `${order.customer.address.city}` returns `""` if any step is null — no defensive null-checks needed
- Use `<c:out value="${val}" />` rather than bare `${val}` whenever displaying data that came from user input or a database
- Set `isELIgnored="true"` only on legacy JSP 1.x pages being migrated — never on new pages

QUICK SUMMARY

- `${expr}` — immediate EL; evaluates and outputs during page rendering
- `.` operator — calls getter; `[]` operator — index/dynamic key access
- Null-safe — null mid-chain returns `""`, not NPE
- `\${` — escape to print literal dollar-brace
- Always escape user data — use `<c:out>` or `fn:escapeXml()`

EL Implicit Objects

pageScope · requestScope · sessionScope · applicationScope · param · paramValues · header · cookie · initParam · pageContext

EL'S BUILT-IN MAP OBJECTS

EL provides eleven implicit objects — all available inside `${}` without any declaration. They are all **read-only Maps** (except `pageContext`, which is the `PageContext` object). The scope maps give direct access to attributes in each scope; the others give access to request parameters, headers, cookies, and init params. They mirror what the Java implicit objects provide but without scriptlets.

KEY FACTS

- All EL implicit objects except `pageContext` are `Map` instances
- `param` returns a single String; `paramValues` returns `String[]`
- `header` / `headerValues` — same pattern as param/paramValues
- `cookie` values are `Cookie` objects — access value via `${cookie.name.value}`
- `initParam` reads `<context-param>` from web.xml — same as `application.getInitParameter()`

NOTES

→ `param.name` — single value; equivalent to `request.getParameter("name")`

→ `paramValues.name` — `String[]`; use `[0]` to get first element

→ `header["X-Name"]` — bracket required for headers with hyphens

→ `cookie.name` — returns the `Cookie` object; chain `.value` for the string value

→ `initParam.key` — reads `<context-param>`; dot notation only if key has no special chars

→ `pageContext` — the only non-Map EL implicit object; gives access to the full PageContext API

PREFER / AVOID

✗ ~~`${param.x}` in views~~ → ✓ **controller-validated model values**

✗ ~~`${cookie.name.value}` for logic~~ → ✓ **read cookies in the controller**

SECURITY CHECK

- `param`, `header` and `cookie` are user input
- Validate in the controller; escape on output

EL IMPLICIT OBJECT USAGE

```
<%-- ── Scope maps ──────────────────────────────────────── --%>
${pageScope.localVar}            <%-- page-scope attribute --%>
${requestScope.products}         <%-- request-scope attribute --%>
${sessionScope.currentUser}      <%-- session-scope attribute --%>
${applicationScope.appVersion}   <%-- application-scope attribute --%>

<%-- ── Request parameters ──────────────────────────────── --%>
${param.q}                       <%-- request.getParameter("q") --%>
${param["page-num"]}             <%-- param with hyphen in name --%>
${paramValues.ids}               <%-- String[] — multi-value param --%>
${paramValues.ids[0]}            <%-- first value of multi-param --%>

<%-- ── Request headers ─────────────────────────────────── --%>
${header["User-Agent"]}          <%-- bracket required (hyphen) --%>
${header["Accept-Language"]}
${headerValues["Accept"][0]}     <%-- first Accept header value --%>

<%-- ── Cookies ─────────────────────────────────────────── --%>
${cookie.JSESSIONID.value}       <%-- cookie value --%>
${cookie.theme.value}            <%-- custom cookie --%>
${cookie.theme.maxAge}           <%-- cookie max-age --%>

<%-- ── Context init params (web.xml <context-param>) ──── --%>
${initParam["support.email"]}
${initParam.appName}

<%-- ── pageContext object ──────────────────────────────── --%>
${pageContext.request.contextPath}   <%-- /myapp --%>
${pageContext.request.method}        <%-- GET / POST --%>
${pageContext.session.id}            <%-- JSESSIONID --%>
${pageContext.servletContext.serverInfo} <%-- e.g. Apache Tomcat/10.1 --%>
```

MODERN EQUIVALENT

- Spring: `@RequestParam`, `@RequestHeader`, `@CookieValue`

ALL EL IMPLICIT OBJECTS

NAME	TYPE	CONTAINS
pageScope	Map	Page-scope attributes
requestScope	Map	Request-scope attributes
sessionScope	Map	Session-scope attributes
applicationScope	Map	Application-scope attributes
param	Map<String,String>	First value of each request param
paramValues	Map<String,String[]>	All values of each request param
header	Map<String,String>	First value of each request header
headerValues	Map<String,String[]>	All values of each request header
cookie	Map<String,Cookie>	Cookies by name
initParam	Map<String,String>	context-param entries
pageContext	PageContext	Full PageContext object

⚠ COMMON MISTAKES

- Writing `${cookie.name}` and expecting the string value — it returns the `Cookie` object; you need `${cookie.name.value}`
- Using dot notation for headers: `${header.User-Agent}` — EL parses the hyphen as subtraction; always use `${header["User-Agent"]}`
- Displaying `${param.input}` directly — EL does not escape HTML; user-supplied values can inject script tags

✓ TIPS

- Use `${pageContext.request.contextPath}` in every href and src attribute — it is the safest and cleanest way to build portable links in JSPs
- Prefer `${requestScope.key}` over bare `${key}` when you know the scope — prevents accidental resolution from a wider scope
- Read `${initParam.configKey}` in JSPs for display-only config values; keep business config in Java service classes

QUICK SUMMARY

- `param / paramValues` — request params; single vs multi-value
- `header["Name"]` — use brackets; headers contain hyphens
- `cookie.name.value` — chain .value to get string from Cookie object
- `initParam.key` — reads <context-param> from web.xml
- `pageContext` — sole non-Map; full PageContext API available

EL Operators & Type Coercion

EL OPERATOR SET

EL supports a full set of operators that work entirely within ${} expressions — no Java scriptlet required. Operators come in keyword and symbol forms: eq and == are identical. The empty operator tests for null, empty string, zero-length array/collection, or empty map — a single test that replaces multiple null-checks. The ternary ? : is useful for inline conditionals in attribute values.

KEY FACTS

- Keyword aliases exist for XML-unsafe symbols: `lt`, `gt`, `le`, `ge`, `eq`, `ne`, `and`, `or`, `not`
- `empty x` is true when: x is null, `""`, empty array, empty Collection, or empty Map
- Division by zero returns `Infinity` for doubles, throws `ArithmeticException` for integers
- EL 3.0 added `+=` string concatenation operator: `${"Hello" += " World"}`
- Precedence: `[] .` → unary → arithmetic → relational → logical → ternary → assignment

EL OPERATOR EXAMPLES

```
<%-- ── Arithmetic ──────────────────────────────────────── --%>
${5 + 3}          <%-- 8 --%>
${10 - 4}         <%-- 6 --%>
${3 * 4}          <%-- 12 --%>
${10 / 4}         <%-- 2.5 --%>
${10 div 4}       <%-- 2.5   (keyword alias) --%>
${10 % 3}         <%-- 1 --%>
${10 mod 3}       <%-- 1     (keyword alias) --%>

<%-- ── Relational ──────────────────────────────────────── --%>
${price == 0}     <%-- true/false --%>
${price eq 0}     <%-- identical (XML-safe alias) --%>
${price != 0}     <%-- not equal --%>
${price ne 0}     <%-- identical --%>
${price > 100}    <%-- greater than --%>
${price gt 100}   <%-- identical --%>
${price >= 100}   <%-- greater or equal --%>
${price ge 100}   <%-- identical --%>
${price < 10}     <%-- less than --%>
${price lt 10}
${price le 10}    <%-- less or equal --%>

<%-- ── Logical ─────────────────────────────────────────── --%>
${a && b}         <%-- logical AND --%>
${a and b}        <%-- identical --%>
${a || b}         <%-- logical OR --%>
${a or b}         <%-- identical --%>
${!active}        <%-- logical NOT --%>
${not active}     <%-- identical --%>

<%-- ── empty operator ──────────────────────────────────── --%>
${empty list}         <%-- true if null or size==0 --%>
${not empty list}     <%-- true if non-null and non-empty --%>
${empty param.q}      <%-- true if param missing or blank --%>

<%-- ── Ternary ─────────────────────────────────────────── --%>
${user.active ? "Active" : "Inactive"}
${empty cart ? "Empty" : cart.size += " items"}

<%-- ── instanceof (EL 3.0) ─────────────────────────────── --%>
${product instanceof com.example.DigitalProduct}
```

OPERATOR NOTES

- → `==` / `eq` — structural equality; uses `.equals()` on objects
- → `empty` — single keyword tests null, blank, and empty collections at once
- → `and` / `or` / `not` — XML-safe alternatives; use in tag attributes to avoid escaping `&&`
- → `lt` / `gt` / `le` / `ge` — use in XML attribute values where `<` and `>` need escaping
- → `? :` — ternary; both branches must be valid EL expressions
- → `+=` — EL 3.0 string concatenation; not available in EL 2.x

OPERATOR REFERENCE

CATEGORY	SYMBOL	KEYWORD
Addition	+	—
Subtraction	-	—
Multiply	*	—
Divide	/	div
Modulo	%	mod
Equal	==	eq
Not equal	!=	ne
Less than	<	lt
Greater than	>	gt
Less or eq	<=	le
Greater or eq	>=	ge
AND	&&	and
OR	||	or
NOT	!	not
Empty test	—	empty
Ternary	?:	—

EMPTY OPERATOR TRUTH TABLE

VALUE	EMPTY RESULT
null	true
""	true
[] (empty array)	true
[] (empty List)	true
{} (empty Map)	true
"hello"	false
[item] (non-empty)	false
0 (zero)	false
false	false

MODERN EQUIVALENT

- Thymeleaf / SpEL use the same operators

PREFER / AVOID

- ✗ ~~`${a == null || a == ''}`~~ → ✓ `${empty a}`
- ✗ ~~complex logic in EL~~ → ✓ **a computed bean property**

⚠ COMMON MISTAKES

- Using `<` and `>` inside tag attributes — XML parsers reject them; use `lt` and `gt` inside attribute values
- Expecting `empty 0` to be true — zero is not empty in EL; only null, `""`, and empty collections are
- Using `==` to compare strings and getting unexpected results — EL uses `.equals()` semantics, which is correct, but ensure types match; `"5" == 5` coerces and evaluates true

✓ TIPS

- Use `${not empty list}` rather than `${list != null && list.size() > 0}` — `empty` covers all cases in one operator
- Always use keyword aliases (`lt`, `gt`, `and`, `or`) inside XML attribute values — avoids `&` escaping and parse errors
- For complex conditionals in EL, simplify by computing a boolean in the servlet and passing it as a request attribute — keep EL expressions short

QUICK SUMMARY

- `eq / ne / lt / gt / le / ge` — keyword aliases for XML safety
- `and / or / not` — keyword aliases for `&& || !`
- `empty x` — true if null, "", or empty collection/map
- `? :` — ternary conditional inside EL
- Use keyword forms inside tag attribute values to avoid XML escaping

Sheet 16

EL Collection & Bean Access

list[index] · map[key] · bean.property · nested access · EL 3.0 streams · lambda in EL

NAVIGATING COLLECTIONS IN EL

EL can traverse any Java collection or bean without a scriptlet. Lists and arrays use bracket notation with an integer index. Maps use bracket notation with a string key. JavaBeans use the dot or bracket operator against the property name, which EL resolves via the `getX()` / `isX()` convention. All access is null-safe — a null anywhere in a chain returns empty string rather than throwing.

KEY FACTS

- List/array index is zero-based: `${list[0]}` is the first element
- Map key can be dynamic: `${map[varName]}` looks up the value of `varName` as the key
- Bean property `${bean.name}` calls `bean.getName()` ; boolean uses `isName()`
- Chains are null-safe at every step: `${order.customer.address.city}` returns `""` if any link is null
- EL 3.0 (Jakarta EE 7+) added stream API and lambda support for filtering collections

ACCESS NOTES

- → `list[n]` — zero-based index; works on arrays, List, LinkedList
- → `map["key"]` — bracket required for keys with dots or hyphens
- → `map[var]` — dynamic key; `var` is evaluated as a string first
- → `bean.prop` — calls `getProp()` ; must follow JavaBean naming
- → `bean.boolProp` — calls `isBoolProp()` for boolean properties
- → `.stream().filter().toList()` — EL 3.0; Jakarta EE 7+ containers only
- → `st.index` — 0-based; `st.count` — 1-based; both from `varStatus`

MODERN EQUIVALENT

- Thymeleaf: `${list[0]}` , `${map['k']}` , `${bean.prop}`

PREFER / AVOID

- ✗ ~~EL 3.0 streams / lambdas in views~~ → **✓ prepare data in Java**
- ✗ ~~`map["key"]` typos~~ → **✓ beans with real getters**

COLLECTION ACCESS PATTERNS

```
<%-- ── List / array ──────────────────────────────────────── --%>
${products[0].name}          <%-- first product's name --%>
${products[0]["name"]}       <%-- identical --%>
${products[lastIndex]}       <%-- dynamic index from EL var --%>

<%-- ── Map ──────────────────────────────────────────────── --%>
${labels["submit.button"]}  <%-- key with dot — bracket required --%>
${config["api.url"]}
${config[selectedKey]}       <%-- dynamic key lookup --%>

<%-- ── JavaBean ─────────────────────────────────────────── --%>
${user.firstName}            <%-- calls user.getFirstName() --%>
${user.active}               <%-- calls user.isActive() (boolean) --%>
${user.address.city}         <%-- nested bean chain --%>
${user.orders[0].total}      <%-- bean → list → bean property --%>

<%-- ── EL 3.0 stream API ──────────────────────────────────── --%>
${products.stream()
    .filter(p -> p.price lt 50)
    .map(p -> p.name)
    .toList()}

<%-- ── Iterate collection — use JSTL (preferred) ──────────── --%>
<c:forEach items="${products}" var="p" varStatus="st">
    <tr class="${st.index % 2 == 0 ? 'even' : 'odd'}">
        <td>${st.count}</td>
        <td><c:out value="${p.name}" /></td>
        <td>${p.price}</td>
    </tr>
</c:forEach>

<%-- ── Map iteration ──────────────────────────────────────── --%>
<c:forEach items="${config}" var="entry">
    <p>${entry.key} = ${entry.value}</p>
</c:forEach>
```

ACCESS PATTERN REFERENCE

DATA TYPE	EL ACCESS	JAVA EQUIVALENT
`List<T>`	${list[2]}	list.get(2)
`T[]`	${arr[2]}	arr[2]
`Map<String,V>`	${map.key}	map.get("key")
`Map (special key)`	${map["a.b"]}	map.get("a.b")
`JavaBean prop`	${bean.name}	bean.getName()
`Boolean bean prop`	${bean.active}	bean.isActive()
`Nested bean`	${a.b.c}	a.getB().getC()
`Bean → List`	${u.orders[0]}	u.getOrders().get(0)

C:FOREACH VARSTATUS PROPERTIES

PROPERTY	TYPE	DESCRIPTION
`index`	int	Current iteration (0-based)
`count`	int	Current iteration (1-based)
`first`	boolean	True on first iteration
`last`	boolean	True on last iteration
`begin`	int	begin attribute value
`end`	int	end attribute value
`step`	int	step attribute value

⚠ COMMON MISTAKES

- Using `${map.key.with.dots}` — EL navigates three levels of beans; use `${map["key.with.dots"]}` for a map key that contains dots
- Expecting `${list.size()}` to work — EL dot notation calls getters, not arbitrary methods; use `${fn:length(list)}` or `<c:forEach>` with `varStatus`
- Forgetting that `boolean` properties must use `is` prefix — if the getter is `getActive()` instead of `isActive()` , `${bean.active}` may return null

✓ TIPS

- Use `${fn:length(collection)}` to get collection size — it works on arrays, Lists, Maps, and strings uniformly
- Take advantage of EL's null-safety in deep chains — `${order.customer.address.city}` is safe without null checks; a null mid-chain silently returns `""`
- Avoid EL 3.0 stream API in template markup — it reduces readability; do filtering in the servlet and pass a pre-filtered list as a request attribute

QUICK SUMMARY

- `${list[0]}` — zero-based list/array access
- `${map["dotted.key"]}` — bracket required for special-char keys
- `${bean.prop}` — calls getter via JavaBean convention
- Null-safe chains — any null returns `""` , not NPE
- `fn:length(col)` — get size of any collection type

EL Functions & Custom Functions

fn:length · fn:substring · fn:contains · fn:split · fn:join · fn:escapeXml · custom static method

CALLING FUNCTIONS IN EL

EL does not allow arbitrary method calls — only **registered EL functions** declared in a TLD. JSTL provides the `fn:` library of string and collection utility functions. You can also map your own `public static` Java methods to EL functions via a TLD. Functions are called as `${prefix:name(arg1, arg2)}` inside any EL expression.

KEY FACTS

- Declare the fn: library: `<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>`
- EL functions must be `public static` methods in a Java class registered in a TLD
- `fn:length()` works on String, arrays, Collections, and Maps
- `fn:escapeXml()` converts `<` `>` `&` `"` `'` to HTML entities — always use on user input
- String indices in `fn:` functions are zero-based

FN: FUNCTION USAGE

```
<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>

<%-- ── String length / test ──────────────────────────── --%>
${fn:length("hello")}           <%-- 5 --%>
${fn:length(productList)}       <%-- list size --%>
${fn:contains("foobar", "foo")} <%-- true --%>
${fn:containsIgnoreCase("Hello","hello")} <%-- true --%>
${fn:startsWith("index.jsp", "index")}    <%-- true --%>
${fn:endsWith("file.jsp", ".jsp")}        <%-- true --%>
${fn:indexOf("foobar", "bar")}  <%-- 3 --%>

<%-- ── Substrings ────────────────────────────────────── --%>
${fn:substring("Hello World", 6, 11)}    <%-- "World" --%>
${fn:substringBefore("foo@bar", "@")}    <%-- "foo" --%>
${fn:substringAfter("foo@bar", "@")}     <%-- "bar" --%>

<%-- ── Transform ─────────────────────────────────────── --%>
${fn:toUpperCase("hello")}       <%-- "HELLO" --%>
${fn:toLowerCase("HELLO")}       <%-- "hello" --%>
${fn:trim("  hi  ")}             <%-- "hi" --%>
${fn:replace("a-b-c", "-", "_")} <%-- "a_b_c" --%>

<%-- ── Split / Join ──────────────────────────────────── --%>
<%-- fn:split returns String[] --%>
<c:set var="parts" value="${fn:split('a,b,c', ',')}" />
${parts[0]}                      <%-- "a" --%>
${fn:join(parts, " | ")}         <%-- "a | b | c" --%>

<%-- ── Escape (ALWAYS for user data) ─────────────────── --%>
${fn:escapeXml(param.comment)}   <%-- <script> → safe --%>

<%-- ── Custom EL function usage (after TLD registration) ── --%>
<%@ taglib prefix="myutil" uri="/WEB-INF/tlds/myutil.tld" %>
${myutil:formatCurrency(product.price, "USD")}
```

CUSTOM FUNCTION TLD ENTRY

- → `fn:length(obj)` — String char count or collection size
- → `fn:substring(s, start, end)` — indices zero-based, end exclusive
- → `fn:split(str, delim)` — returns `String[]`; iterate with `c:forEach`
- → `fn:join(arr, sep)` — joins String[] with separator
- → `fn:escapeXml(str)` — escapes 5 XML/HTML special chars; use on all untrusted output
- → Custom function — must be `public static`, registered in TLD `<function>` element

COMPLETE FN: FUNCTION REFERENCE

FUNCTION	RETURNS	NOTES
`fn:length(obj)`	int	String, Collection, Map, array
`fn:contains(s,sub)`	boolean	Case-sensitive
`fn:containsIgnoreCase(s,sub)`	boolean	—
`fn:startsWith(s,pre)`	boolean	—
`fn:endsWith(s,suf)`	boolean	—
`fn:indexOf(s,sub)`	int	-1 if not found
`fn:substring(s,b,e)`	String	0-based, end exclusive
`fn:substringBefore(s,sub)`	String	Before first match
`fn:substringAfter(s,sub)`	String	After first match
`fn:replace(s,b,r)`	String	All occurrences
`fn:split(s,delim)`	String[]	—
`fn:join(arr,sep)`	String	String[] only
`fn:toUpperCase(s)`	String	—
`fn:toLowerCase(s)`	String	—
`fn:trim(s)`	String	Leading/trailing whitespace
`fn:escapeXml(s)`	String	Escapes < > & " '

MODERN EQUIVALENT

- Thymeleaf `#strings` utilities · JS string methods in React

PREFER / AVOID

- ✗ ~~custom functions for display logic~~ → ✓ **format in the controller**
- ✗ ~~`fn:substring` for truncation chains~~ → ✓ **one helper method**

SECURITY CHECK

- `fn:escapeXml` only covers HTML text
- Use `fn:escapeXml` / `c:out` in HTML; not enough for JS or URLs

CUSTOM EL FUNCTION — TLD ENTRY

TLD ELEMENT	VALUE
`<function>`	Container element
`<name>`	EL name: formatCurrency
`<function-class>`	com.example.ELFunctions
`<function-signature>`	java.lang.String formatCurrency(java.math.BigDecimal, java.lang.String)

JAVA CLASS RULE	REQUIREMENT
`Method access`	Must be public
`Method modifier`	Must be static
`Return type`	Any; coerced by EL
`Parameters`	Any types; EL coerces args

⚠ COMMON MISTAKES

- Forgetting `fn:escapeXml()` on user-supplied output — bare `${param.x}` is XSS-vulnerable; always wrap with `${fn:escapeXml(param.x)}` or use `<c:out>`
- Passing a `List` to `fn:join()` — it only accepts `String[]`; convert with `fn:split(fn:join(...), ",")` or iterate with `c:forEach`
- Calling `${fn:substring(s, 0, fn:length(s))}` when you want a trimmed copy — just use `${fn:trim(s)}`

✓ TIPS

- Use `${fn:escapeXml(value)}` as the default for any output that may contain user data — make it a habit, not an afterthought
- Check empty before using string functions: `${not empty s ? fn:toUpperCase(s) : ''}` — some fn: functions throw on null input
- Register project-wide utility functions (currency formatting, date formatting) as custom EL functions — keeps JSPs clean and functions testable as plain Java statics

QUICK SUMMARY

- `${fn:length(x)}` — size of String, List, Map, or array
- `${fn:escapeXml(s)}` — escape HTML entities; use on all user data
- `${fn:split(s,d)}` → String[]; `${fn:join(arr,sep)}` → String
- `${fn:contains / startsWith / endsWith}` — boolean string tests
- Custom functions — `public static` methods registered in TLD

jsp:include & jsp:forward

RUNTIME DISPATCH ACTIONS

<jsp:include> and <jsp:forward> are standard action tags that use the servlet container's RequestDispatcher at *request time*. Unlike the <%@ include %> directive (translation time), these resolve the path dynamically — the page attribute can contain an EL expression. include merges another resource's output into the current response; forward hands control to another resource and stops the current page.

KEY FACTS

- `<jsp:include>` runs the included resource as a separate servlet; it gets its own page scope
- `<jsp:forward>` immediately halts the current page — no more output after it
- The `page` attribute supports EL expressions and runtime values — unlike `<%@ include %>`
- `<jsp:param>` children pass additional request parameters to the dispatched resource
- `flush="true"` on `<jsp:include>` flushes the buffer before including — response headers are then committed

CODE NOTES

- → `page` — context-relative path or EL expression; resolved at request time
- → `<jsp:param>` — adds to (not replaces) the current request's parameters; readable via `request.getParameter()` in the target
- → `flush="true"` — commits buffer; prevents setting headers after include; default is `false`
- → `<jsp:forward>` — uses `RequestDispatcher.forward()`; execution of current JSP stops immediately
- → Request attributes survive both include and forward — set them on `request` before dispatching

MODERN EQUIVALENT

- Thymeleaf `th:replace` / Spring `forward:` / `redirect:`

PREFER / AVOID

- ✗ ~~`jsp:forward` from pages~~ → ✓ **forward in the servlet / controller**
- ✗ ~~dynamic `page="${param.p}"`~~ → ✓ **a fixed allow-list**

SECURITY CHECK

- `jsp:include page="${param.p}"` = file disclosure
- Never build include paths from input

INCLUDE & FORWARD SYNTAX

```
<%-- ── jsp:include — merges output at runtime ──────────────── --%>

<%-- Static path --%>
<jsp:include page="/WEB-INF/jsp/header.jsp" />

<%-- Dynamic path via EL --%>
<jsp:include page="${widget.templatePath}" />

<%-- With extra parameters --%>
<jsp:include page="/WEB-INF/jsp/sidebar.jsp">
    <jsp:param name="section" value="news" />
    <jsp:param name="limit"   value="5" />
</jsp:include>

<%-- flush=true: commit buffer before including --%>
<%-- Use only when you are certain no headers need setting after --%>
<jsp:include page="/WEB-INF/jsp/footer.jsp" flush="true" />

<%-- ── jsp:forward — delegate to another resource ──────────── --%>

<%-- Forward to a JSP --%>
<jsp:forward page="/WEB-INF/jsp/result.jsp" />

<%-- Forward with extra params (readable via request.getParameter) --%>
<jsp:forward page="/WEB-INF/jsp/error.jsp">
    <jsp:param name="code"    value="404" />
    <jsp:param name="message" value="Not Found" />
</jsp:forward>

<%-- Conditional forward (common pattern) --%>
<%
    if (session.getAttribute("user") == null) {
%>
<jsp:forward page="/WEB-INF/jsp/login.jsp" />
<%
    }
%>
```

INCLUDE VS FORWARD COMPARISON

FEATURE	JSP:INCLUDE	JSP:FORWARD
Output merged	✔	✖ replaces
Current page continues	✔	✖ stops
Page scope shared	✖ separate	✖ separate
Request attrs shared	✔	✔
Dynamic path	✔	✔
jsp:param children	✔	✔

DIRECTIVE VS ACTION INCLUDE

FEATURE	<%@ INCLUDE %>	<JSP:INCLUDE>
When resolved	Translation	Request time
Dynamic path	✖	✔
Separate page scope	✖ merged	✔
Change detection	Manual	Automatic
Extra params	✖	✔ via jsp:param
Best for	Static fragments	Dynamic components

⚠ COMMON MISTAKES

- Writing output after `<jsp:forward>` — the forward takes effect immediately but the remaining JSP code still compiles and may produce buffered output that garbles the response
- Using `flush="true"` and then trying to set response headers or cookies — once flushed, the response is committed and headers cannot be changed
- Expecting included pages to inherit page-scope variables — each `<jsp:include>` target has its own page scope; use request attributes to share data

✓ TIPS

- After `<jsp:forward>`, add a `<% return; %>` scriptlet — makes the intent explicit and silences IDE warnings about unreachable code
- Use `<jsp:include>` for reusable widgets (pagination bars, sidebars) — they can be tested independently as standalone JSPs
- Set request attributes before forwarding/including rather than using `<jsp:param>` for complex data — params are strings only; attributes carry typed objects

QUICK SUMMARY

- `<jsp:include>` — inserts resource output; current page continues
- `<jsp:forward>` — delegates entirely; current page stops
- `page` attr — supports runtime EL expressions
- `<jsp:param>` — adds string params; use request attrs for objects
- Request attributes travel across both include and forward

Sheet 19

jsp:useBean & Property Actions

jsp:useBean · id · class · scope · jsp:setProperty · property="*" · jsp:getProperty · param auto-bind

BEAN WIRING WITHOUT SCRIPTLETS

`<jsp:useBean>` locates or creates a JavaBean in a specified scope, then assigns it to a scripting variable. `<jsp:setProperty>` sets one or more bean properties — with `property="*"` it automatically maps all matching request parameters to bean properties. `<jsp:getProperty>` reads and outputs a single property. These actions bridge request parameters to JavaBeans without Java scriptlets.

KEY FACTS

- `<jsp:useBean>` looks in the specified scope first — only creates a new instance if none exists
- The bean class must have a public no-arg constructor and follow JavaBean conventions
- `property="*"` auto-maps request params to bean properties by matching name; type conversion is automatic
- The bean body executes only when a *new* bean was created — not when reusing an existing one
- Modern practice: prefer setting beans in the servlet via `request.setAttribute()` and reading via EL in JSP

USEBEAN, SETPROPERTY, GETPROPERTY

```
<%-- ── useBean: find or create a bean in a scope ──────────── --%>

<%-- Simplest form: create in page scope --%>
<jsp:useBean id="user" class="com.example.User" scope="page" />

<%-- Reuse from request scope (set by servlet) --%>
<jsp:useBean id="order" class="com.example.Order" scope="request" />

<%-- Create in session scope; body runs only on new creation --%>
<jsp:useBean id="cart" class="com.example.Cart" scope="session">
    <% cart.init(request.getParameter("userId")); %>
</jsp:useBean>

<%-- type: use interface/superclass as the variable type --%>
<jsp:useBean id="svc" type="com.example.Service"
             class="com.example.ServiceImpl" scope="application" />

<%-- ── setProperty ──────────────────────────────────────── --%>

<%-- Map all request params to matching bean properties --%>
<jsp:setProperty name="user" property="*" />

<%-- Set one property from matching request param --%>
<jsp:setProperty name="user" property="email" />

<%-- Set one property from a specific param name --%>
<jsp:setProperty name="user" property="email" param="emailAddr" />

<%-- Set one property to a literal value --%>
<jsp:setProperty name="user" property="role" value="GUEST" />

<%-- ── getProperty ──────────────────────────────────────── --%>
<p>Name: <jsp:getProperty name="user" property="firstName" /></p>
<%-- EL equivalent (preferred): ${user.firstName} --%>
```

ATTRIBUTE NOTES

- → `id` — scripting variable name; also the scope attribute key
- → `class` — fully qualified concrete class; must have no-arg constructor
- → `type` — declares the variable type (interface/superclass); use with `class` or alone when bean already exists in scope
- → `scope` — page | request | session | application; default is page
- → `property="*"` — auto-bind: maps each request param to setter by name with type coercion
- → `param` — use when param name differs from property name
- → Bean body — the content between open/close tags runs only when a new instance is created

MODERN EQUIVALENT

- Spring `@ModelAttribute` data binding with `@InitBinder` allow-lists

PREFER / AVOID

- ✗ ~~`jsp:useBean` + `property="*"`~~ → **✓ bind in the controller, validated**
- ✗ ~~beans created in views~~ → **✓ model objects from Java**

SECURITY CHECK

- `property="*"` sets every matching field (mass assignment)
- Bind explicit properties only

JSP:USEBEAN ATTRIBUTES

ATTRIBUTE	REQUIRED	DESCRIPTION
`id`	✔	Scripting var and scope key
`class`	*	Concrete class to instantiate
`type`	*	Variable's declared type
`scope`	✖	page|request|session|application
`beanName`	—	For Beans.instantiate() lookup

JSP:SETPROPERTY COMBINATIONS

PROPERTY	PARAM	VALUE	EFFECT
`*`	—	—	All params → matching props
`propName`	—	—	param[propName] → prop
`propName`	paramName	—	param[paramName] → prop
`propName`	—	literal	Literal string → prop

⚠ COMMON MISTAKES

- Using `property="*"` without validating request parameters first — it blindly maps any param to a setter, allowing attackers to set unexpected properties (mass-assignment vulnerability)
- Assuming the bean body always runs — it only executes when a new instance is created; if the bean already exists in the scope, the body is skipped silently
- Using `type` alone (without `class`) when no bean exists in scope — throws an `InstantiationException` because there is nothing to instantiate

✓ TIPS

- In modern MVC, set beans in the servlet via `request.setAttribute()` and read them in JSP via EL — `<jsp:useBean>` is mainly useful in simpler or legacy page-centric patterns
- Use `type` instead of `class` when the bean is already placed in scope by a servlet — avoids unnecessary second-class creation attempt
- Never use `property="*"` for user-facing forms without a whitelist of allowed properties — always explicitly name each property

QUICK SUMMARY

- `<jsp:useBean>` — find or create bean in scope; id becomes EL variable
- Bean body — runs only when a new instance is created
- `property="*"` — auto-bind all matching request params to setters
- `<jsp:getProperty>` — outputs one property; EL preferred
- Modern pattern: set in servlet → read via EL in JSP

Other Standard Actions

BEYOND INCLUDE, FORWARD & USEBEAN

JSP defines several additional standard actions for dynamic XML/HTML generation. `<jsp:element>` creates an element with a computed tag name at runtime. `<jsp:attribute>` sets an attribute whose value may contain EL or JSP content. `<jsp:body>` provides the body content of a parent action. `<jsp:text>` wraps template text for use inside action bodies. `<jsp:output>` controls the XML declaration and DOCTYPE in JSP documents.

KEY FACTS

- `<jsp:element>` is most useful when the tag name itself is dynamic (e.g. driven by a variable)
- `<jsp:attribute>` inside a custom tag allows attribute values that contain nested JSP expressions
- `<jsp:body>` provides body content to a tag that calls `invoke()` or `getJspBody()`
- `<jsp:text>` outputs literal text without the container interpreting template content — useful in JSP documents (.jspx)
- `<jsp:output>` is only valid in JSP documents (XML syntax, .jspx); it controls the XML/DOCTYPE declaration

STANDARD ACTION USAGE

```
<%-- ── jsp:element — dynamic tag name ──────────────────── --%>
<jsp:element name="${headingLevel}">
  <jsp:attribute name="class">title</jsp:attribute>
  <jsp:body>${section.title}</jsp:body>
</jsp:element>
<%-- renders: <h2 class="title">Introduction</h2> if headingLevel="h2" --%>

<%-- ── jsp:attribute — complex attribute value ─────────── --%>
<jsp:element name="a">
  <jsp:attribute name="href">
    ${pageContext.request.contextPath}/item/${item.id}
  </jsp:attribute>
  <jsp:body>${item.name}</jsp:body>
</jsp:element>

<%-- ── jsp:attribute inside a custom tag ──────────────── --%>
<my:panel>
  <jsp:attribute name="title">
    <strong>${user.name}</strong>'s Dashboard
  </jsp:attribute>
  <jsp:body>
    <p>Welcome back!</p>
  </jsp:body>
</my:panel>

<%-- ── jsp:text — literal output in JSP documents ──────── --%>
<jsp:text>This text is output verbatim.</jsp:text>

<%-- ── jsp:output — XML declaration (jspx files only) ──── --%>
<jsp:output omit-xml-declaration="true" />
<jsp:output doctype-root-element="html"
            doctype-system="about:legacy-compat" />
```

NOTES

- → `<jsp:element name="${var}">` — name evaluated at request time; generates opening/closing tags
- → `<jsp:attribute name="x">` — sets attribute on parent element; body can contain JSP/EL
- → `<jsp:body>` — provides body to parent tag or element; required when sibling `<jsp:attribute>` is present
- → `<jsp:text>` — wraps whitespace or literal text; prevents container from trimming it in JSP documents
- → `<jsp:output>` — controls XML prolog; only valid in .jspx files, not .jsp files
- → `<jsp:root>` — root element of a JSP document; carries namespace declarations

PREFER / AVOID

✗ ~~JSPX documents for XML~~ → **✔ a serializer (JAXB / Jackson)**

ALL STANDARD ACTIONS REFERENCE

ACTION	PURPOSE
`jsp:include`	Include resource output at runtime
`jsp:forward`	Delegate request to another resource
`jsp:param`	Pass extra parameter to include/forward
`jsp:useBean`	Find or create JavaBean in a scope
`jsp:setProperty`	Set one or more bean properties
`jsp:getProperty`	Output one bean property
`jsp:element`	Generate XML/HTML element with dynamic name
`jsp:attribute`	Set attribute with JSP/EL content
`jsp:body`	Provide body to parent tag/element
`jsp:text`	Output literal text verbatim
`jsp:output`	Control XML declaration (.jspx only)
`jsp:root`	Root element of JSP document (.jspx)
`jsp:plugin`	Deprecated — Java applet embedding

JSP SYNTAX (.JSP) VS JSP DOCUMENT (.JSPX)

FEATURE	.JSP	.JSPX
Syntax	Tag-based	Well-formed XML
`<% %>` scriptlets	✔	✖
`jsp:root` required	✖	✔
`jsp:output` valid	✖	✔
`jsp:text` needed	Rarely	Often
`xmlns` on root	✖	✔

MODERN EQUIVALENT

- Thymeleaf XML mode / JSON views (`@ResponseBody`)

⚠ COMMON MISTAKES

- Using `<jsp:output>` in a `.jsp` file — it is only valid in JSP documents (`.jspx`); in a regular JSP it causes a translation error
- Omitting `<jsp:body>` when a sibling `<jsp:attribute>` is present inside `<jsp:element>` — without it the element has no body content and the output is an empty element
- Nesting template text directly inside a custom tag that expects `<jsp:attribute>` / `<jsp:body>` — the container may treat it as body content and not an attribute value

✓ TIPS

- Use `<jsp:element>` when generating elements whose tag name varies by data (e.g., semantic heading levels h1–h6 driven by a depth counter)
- Use `<jsp:attribute>` inside custom tag invocations when the attribute value needs to contain EL expressions or nested JSP content that cannot fit in a plain XML attribute string
- Prefer `.jsp` (standard syntax) over `.jspx` for most projects — `.jspx` 's strict XML well-formedness requirement adds friction without proportional benefit in most web UIs

QUICK SUMMARY

- `<jsp:element name="${var}">` — dynamic element tag name at runtime
- `<jsp:attribute>` — attribute value with nested JSP/EL content
- `<jsp:body>` — required when jsp:attribute siblings are present
- `<jsp:text>` — literal text verbatim; mainly for .jspx files
- `<jsp:output>` — XML declaration control; .jspx only

JSTL Setup & Library Overview

WHAT JSTL IS

The **Jakarta Standard Tag Library (JSTL)** is a set of tag libraries that replaces scriptlets for the most common JSP tasks: iteration, conditionals, output escaping, URL building, number and date formatting, and string functions. JSTL is not bundled with the servlet container — you must add the JAR. JSTL 3.x targets Jakarta EE 9+ (package `jakarta.*`); JSTL 1.2 targets older `javax.*` stacks.

KEY FACTS

- JSTL 3.x artifact: `org.glassfish.web:jakarta.servlet.jsp.jstl:3.0.1`
- Also need the API JAR: `jakarta.servlet.jsp.jstl:jakarta.servlet.jsp.jstl-api:3.0.0`
- Both JARs go in `WEB-INF/lib/` — the container does not supply JSTL
- Jakarta EE 9+ uses short-form URIs: `jakarta.tags.core` etc.
- Legacy javax stacks use `http://java.sun.com/jsp/jstl/core` URIs

MAVEN DEPENDENCIES & TAGLIB DECLARATIONS

```
/* pom.xml — Jakarta EE 10 / JSTL 3.x */
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.1</version>
</dependency>
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>3.0.0</version>
</dependency>
```

```
<%-- Taglib declarations — Jakarta EE 9+ short URIs --%>
<%@ taglib prefix="c"   uri="jakarta.tags.core"      %>
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt"       %>
<%@ taglib prefix="fn"  uri="jakarta.tags.functions" %>
<%@ taglib prefix="sql" uri="jakarta.tags.sql"       %>
<%@ taglib prefix="x"   uri="jakarta.tags.xml"       %>

<%-- Legacy javax URIs (JSTL 1.2 / pre-Jakarta EE 9) --%>
<%@ taglib prefix="c"   uri="http://java.sun.com/jsp/jstl/core"      %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"       %>
<%@ taglib prefix="fn"  uri="http://java.sun.com/jsp/jstl/functions" %>

<%-- Recommended: one shared fragment declares all taglibs --%>
<%-- taglibs.jspf — include this at top of every JSP     --%>
<%@ include file="/WEB-INF/fragments/taglibs.jspf" %>
```

SETUP NOTES

- → Both JARs required — API jar provides interfaces; impl jar provides the actual tag handlers
- → Short URI `jakarta.tags.core` — works only with JSTL 3.x JARs whose TLD declares this URI
- → Scope: `compile` (or `runtime`) — container does not provide JSTL
- → TLD discovery — container finds TLDs inside `WEB-INF/lib/*.jar` META-INF automatically
- → Shared fragment — declare all taglibs once in `taglibs.jspf`; use static include in every page

MODERN EQUIVALENT

- Thymeleaf ships with Spring Boot starters

PREFER / AVOID

- ✗ ~~JSTL 1.2~~ ~~`javax`~~ ~~JARs on Jakarta EE 10~~ → ✓ **JSTL 3.0 `jakarta` JARs**
- ✗ ~~`sql:`~~ ~~tags~~ → ✓ **DAO / repository in Java**

SECURITY CHECK

- `sql:` tags put SQL and credentials in views
- Data access only in the service layer

JSTL LIBRARY QUICK REFERENCE

PREFIX	URI (JAKARTA)	PURPOSE
`c:`	jakarta.tags.core	Iteration, conditionals, set/remove, URL, output
`fmt:`	jakarta.tags.fmt	Numbers, dates, time zones, i18n messages
`fn:`	jakarta.tags.functions	String functions usable in EL expressions
`sql:`	jakarta.tags.sql	DB queries from JSP (prototype use only)
`x:`	jakarta.tags.xml	XML parsing, XPath, XSLT transforms

JSTL VERSION MATRIX

JSTL	JAKARTA EE	PACKAGE	URI STYLE
`1.1`	J2EE 1.4	javax.*	Full URL
`1.2`	Java EE 5–8	javax.*	Full URL
`2.0`	Jakarta EE 9	jakarta.*	Full URL
`3.0`	Jakarta EE 10	jakarta.*	Short URI

⚠ COMMON MISTAKES

- Using JSTL 3.x short URIs (`jakarta.tags.core`) with JSTL 1.2 JARs — the TLD in the old JAR does not register the short URI; the container reports "cannot find taglib"
- Placing JSTL JARs in Tomcat's `lib/` directory instead of `WEB-INF/lib/` — works locally but breaks when the WAR is deployed to a different container that doesn't have them
- Mixing javax and jakarta JSTL JARs in the same WAR — class-name conflicts cause `ClassCastException` at runtime when tag handlers are instantiated

✓ TIPS

- Create a `taglibs.jspf` fragment with all taglib declarations and include it at the top of every JSP — one change updates all pages
- Avoid `sql:` and `x:` libraries in production — `sql:` bypasses the data access layer; `x:` is rarely needed; both belong in prototypes only
- Check the TLD URI by opening the JSTL JAR and reading `META-INF/*.tld` — the `<uri>` element is the exact string your `taglib` directive must use

QUICK SUMMARY

- JSTL not bundled — add both API and impl JARs to `WEB-INF/lib/`
- JSTL 3.x / Jakarta EE 10 — short URIs: `jakarta.tags.core` etc.
- `c:` core · `fmt:` formatting · `fn:` string functions
- Match JSTL JAR version to Jakarta EE version — don't mix javax/jakarta
- Shared `taglibs.jspf` — declare all prefixes once, include everywhere

JSTL Conditionals

BRANCHING WITHOUT SCRIPTLETS

JSTL core provides two conditional constructs. `<c:if>` is a simple one-way test — there is no `else` branch; use two `<c:if>` tags with opposite conditions, or use `<c:choose>`. `<c:choose>` / `<c:when>` / `<c:otherwise>` is the multi-way branch equivalent of Java's `if/else if/else`. All conditions are EL expressions evaluated to boolean.

KEY FACTS

- `<c:if test="${expr}">` — body renders only when test is true; no else branch
- `var` attribute on `<c:if>` stores the boolean result in scope for later reuse
- `<c:choose>` requires at least one `<c:when>`; `<c:otherwise>` is optional but must be last
- `<c:set>` stores a value in any scope; `<c:remove>` deletes it
- `<c:catch var="e">` catches any Throwable from its body — prevents error page for non-fatal exceptions

TAG NOTES

- → `test` — EL boolean expression; uses EL type coercion (null→false, ""→false)
- → `var` on c:if — saves boolean result; re-use later without re-evaluating
- → `scope` — page|request|session|application; default is page
- → `<c:set var="x" value="${expr}">` — equivalent to scope.setAttribute("x", expr)
- → `<c:set target="${bean}" property="p" value="v">` — calls bean setter
- → `<c:remove>` — removes attribute; if no scope specified, removes from all scopes
- → `<c:catch var="e">` — e is the Throwable; null if no exception occurred

MODERN EQUIVALENT

- Thymeleaf `th:if` / `th:switch`
- React: `cond && <X/>`

PREFER / AVOID

- ✗ ~~nested~~ ~~`c:if`~~ ~~chains~~ → ✓ `c:choose` / `c:when`
- ✗ ~~`c:set`~~ ~~for business logic~~ → ✓ **computed model values**

CONDITIONAL TAG USAGE

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<%-- ── c:if — single branch ──────────────────────────────── --%>
<c:if test="${not empty user}">
  <p>Welcome, ${fn:escapeXml(user.name)}!</p>
</c:if>

<%-- Store test result in a var for reuse --%>
<c:if test="${user.role eq 'ADMIN'}" var="isAdmin" scope="request" />
<c:if test="${isAdmin}"><a href="admin">Admin</a></c:if>

<%-- ── c:choose / c:when / c:otherwise ──────────────────── --%>
<c:choose>
  <c:when test="${order.status eq 'PENDING'}">
    <span class="badge pending">Pending</span>
  </c:when>
  <c:when test="${order.status eq 'SHIPPED'}">
    <span class="badge shipped">Shipped</span>
  </c:when>
  <c:when test="${order.status eq 'DELIVERED'}">
    <span class="badge delivered">Delivered</span>
  </c:when>
  <c:otherwise>
    <span class="badge unknown">${order.status}</span>
  </c:otherwise>
</c:choose>

<%-- ── c:set / c:remove ──────────────────────────────────── --%>
<c:set var="discount" value="${price * 0.1}" scope="request" />
<c:set var="user" property="role" value="GUEST" target="${user}" />
<c:remove var="tempMsg" scope="session" />

<%-- ── c:catch ──────────────────────────────────────────── --%>
<c:catch var="err">
  <%-- risky operation --%>
  <c:import url="${externalUrl}" />
</c:catch>
<c:if test="${not empty err}">
  <p>Could not load content: ${err.message}</p>
</c:if>
```

C:IF ATTRIBUTES

ATTRIBUTE	REQUIRED	DESCRIPTION
`test`	✔	EL boolean expression
`var`	✖	Store result boolean in this name
`scope`	✖	Scope for var; default page

C:SET ATTRIBUTES

ATTRIBUTE	DESCRIPTION
`var`	Attribute name to set in scope
`value`	Value (EL expression or literal)
`scope`	page|request|session|application
`target`	Bean or Map to set property on
`property`	Property name on target bean/map

C:CHOOSE RULES

RULE	DETAIL
`Children only`	c:when and c:otherwise — no other direct children
`c:when count`	One or more required
`c:otherwise`	Optional; must be last child
`Evaluation`	First true c:when wins; rest skipped
`No match`	c:otherwise renders; if absent, nothing renders

⚠ COMMON MISTAKES

- Looking for a `<c:else>` tag — it doesn't exist; use `<c:choose><c:when><c:otherwise>` for if/else
- Placing content directly inside `<c:choose>` that is not a `<c:when>` or `<c:otherwise>` — whitespace is silently ignored but other content causes a tag error
- Using `<c:set>` with both `var` and `target` at the same time — they are mutually exclusive; `var` sets a scope attribute, `target` calls a setter on a bean

✓ TIPS

- Save complex EL conditions with `<c:set var="flag" value="${...}" />` and test `${flag}` — avoids repeating the same long expression in two `<c:if>` tags
- Use `<c:catch>` around `<c:import>` calls to external URLs — network failures should degrade gracefully, not crash the whole page
- Prefer `<c:choose>` over multiple `<c:if>` tags when testing the same variable — semantically clearer and only one branch is evaluated

QUICK SUMMARY

- `<c:if test="${}">` — one-way test; no else; use var to save result
- `<c:choose>` — multi-branch; wraps c:when + c:otherwise
- `<c:set var="x" value="${}">` — set scoped attribute or bean property
- `<c:remove var="x">` — delete attribute from scope
- `<c:catch var="e">` — trap Throwable; prevent error page

JSTL Iteration

c:forEach · items · var · varStatus · begin · end · step · c:forTokens · delims

ITERATING COLLECTIONS WITHOUT SCRIPTLETS

<c:forEach> is the primary JSTL iteration tag. It iterates over any Collection, array, Map, comma-separated string, Iterator, or Enumeration. The varStatus attribute provides loop metadata (index, count, first, last). <c:forTokens> splits a string by one or more delimiters and iterates the tokens — a compact way to loop over delimited data.

KEY FACTS

- `items` accepts: Collection, List, Set, Map, array, Iterator, Enumeration, or comma-separated String
- For Maps, `var` is a `Map.Entry`; access key/value via `${entry.key}` and `${entry.value}`
- `varStatus.index` is 0-based; `varStatus.count` is 1-based
- `begin` / `end` / `step` allow range-based loops without a collection
- `<c:forTokens>` `delims` can contain multiple delimiter characters — any one character matches

C:FOREACH & C:FORTOKENS USAGE

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<%-- ── Basic collection iteration ──────────────────────── --%>
<c:forEach items="${products}" var="p">
  <tr>
    <td><c:out value="${p.name}" /></td>
    <td>${p.price}</td>
  </tr>
</c:forEach>

<%-- ── varStatus: loop metadata ────────────────────────── --%>
<c:forEach items="${orders}" var="o" varStatus="st">
  <tr class="${st.index % 2 == 0 ? 'even' : 'odd'}">
    <td>${st.count}</td>          <%-- 1-based row number --%>
    <td>${o.id}</td>
    <c:if test="${st.first}"><td>← first</td></c:if>
    <c:if test="${st.last}"><td>← last</td></c:if>
  </tr>
</c:forEach>

<%-- ── begin / end / step: numeric range ──────────────── --%>
<c:forEach begin="1" end="10" step="2" var="i">
  ${i}                            <%-- 1 3 5 7 9 --%>
</c:forEach>

<%-- ── Map iteration ──────────────────────────────────── --%>
<c:forEach items="${configMap}" var="entry">
  <p>${entry.key}: ${entry.value}</p>
</c:forEach>

<%-- ── Partial iteration (begin/end on a collection) ──── --%>
<c:forEach items="${allItems}" var="item" begin="0" end="4">
  <%-- first 5 items only --%>
</c:forEach>

<%-- ── c:forTokens: split string by delimiter ─────────── --%>
<c:forTokens items="red,green,blue" delims="," var="color">
  <span class="${color}">${color}</span>
</c:forTokens>

<%-- Multiple delimiters (any one char matches) --%>
<c:forTokens items="a,b;c|d" delims=",;|" var="token">
  ${token}                        <%-- a b c d --%>
</c:forTokens>
```

ATTRIBUTE NOTES

- → `items` — the collection to iterate; omit for numeric range with begin/end
- → `var` — loop variable name; available inside tag body only
- → `varStatus` — loop status object; properties: index, count, first, last, begin, end, step
- → `begin` — start index (0-based for collections); start value for numeric range
- → `end` — end index (inclusive); for collections, limits iterations
- → `step` — increment between iterations; default 1
- → `delims` (forTokens) — string of delimiter characters; each char is an independent delimiter

VARSTATUS PROPERTIES

PROPERTY	TYPE	DESCRIPTION
`index`	int	Current position (0-based)
`count`	int	Iteration number (1-based)
`first`	boolean	True on first iteration
`last`	boolean	True on last iteration
`begin`	Integer	Value of begin attr
`end`	Integer	Value of end attr
`step`	Integer	Value of step attr
`current`	Object	Same as var; the current item

ACCEPTED ITEMS TYPES

TYPE	VAR TYPE INSIDE LOOP
`List / Set / Collection`	Element type
`T[]`	T
`Map<K,V>`	Map.Entry<K,V>
`Iterator`	Element type
`Enumeration`	Element type
`String (comma-sep)`	String token

MODERN EQUIVALENT

- Thymeleaf `th:each="x, stat : ${list}"`
- React: `items.map()`

PREFER / AVOID

- ✗ ~~index loops with `begin` / `end`~~ → ✓ `items` + `varStatus`
- ✗ ~~`c:forTokens` on data~~ → ✓ **a real list from Java**

⚠ COMMON MISTAKES

- Accessing `var` after the `<c:forEach>` closing tag — the variable is removed from scope when the loop ends; save the last value with `<c:set>` inside the loop if needed
- Confusing `varStatus.index` (0-based) with `varStatus.count` (1-based) — using index for display row numbers produces rows starting at 0
- Passing a `List<String>` to `<c:forTokens items>` — `forTokens` requires a String; use `<c:forEach>` for collections

✓ TIPS

- Use `varStatus.first` and `varStatus.last` to add comma separators without an extra `<c:if>`: output the comma before every item except the first
- Use `begin="0" end="4"` on a large collection to render just the first N items — avoids loading all items then hiding them with CSS
- Iterate Maps with `${entry.key}` and `${entry.value}` — no need to call `entrySet()` in a scriptlet first

QUICK SUMMARY

- `<c:forEach items="${list}" var="x">` — iterate any collection or array
- `varStatus` — index (0-based), count (1-based), first, last
- `begin / end / step` — numeric range or collection slice
- Map — var is Map.Entry; use `${entry.key}` / `${entry.value}`
- `<c:forTokens items="a,b" delims=",">` — split string and iterate

JSTL Output, Variables & URLs

c:out · escapeXml · c:url · c:param · c:redirect · c:import · defaultValue

OUTPUT, URLS, AND IMPORTS

`<c:out>` safely outputs a value with automatic HTML escaping — the preferred way to print user-supplied or database data. `<c:url>` builds a context-relative URL, appends session ID if cookies are disabled, and URL-encodes parameters. `<c:redirect>` issues a 302 redirect. `<c:import>` fetches content from a URL (internal or external) and inserts it into the page or stores it in a variable.

KEY FACTS

- `<c:out>` escapes `< > & " '` by default; set `escapeXml="false"` only for trusted HTML content
- `<c:url>` prepends the context path automatically — no need to write `${pageContext.request.contextPath}`
- `<c:url>` also calls `encodeURL()` to append `JSESSIONID` when cookies are disabled
- `<c:param>` inside `<c:url>` URL-encodes parameter values automatically
- `<c:redirect>` must be called before any output — it commits the response

C:OUT, C:URL, C:REDIRECT, C:IMPORT

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<%-- ── c:out — safe HTML output ──────────────────────────── --%>
<c:out value="${user.bio}" />              <%-- escapes HTML entities --%>
<c:out value="${user.bio}" escapeXml="false" /> <%-- trusted HTML --%>
<c:out value="${param.q}" default="(none)" />   <%-- fallback if null --%>

<%-- ── c:url — build context-relative URL ────────────────── --%>
<c:url value="/product/detail" var="detailUrl">
  <c:param name="id"   value="${product.id}" />
  <c:param name="ref"  value="homepage" />
</c:url>
<a href="${detailUrl}">View Product</a>
<%-- renders: /myapp/product/detail?id=42&ref=homepage --%>

<%-- Without var: output inline --%>
<a href="<c:url value='/orders/${order.id}' />">Order</a>

<%-- External URL (left unchanged, just encoded) --%>
<c:url value="https://api.example.com/data" var="apiUrl">
  <c:param name="key" value="${apiKey}" />
</c:url>

<%-- ── c:redirect — 302 response ─────────────────────────── --%>
<c:if test="${empty sessionScope.user}">
  <c:redirect url="/login" />
</c:if>
<c:redirect url="/dashboard">
  <c:param name="msg" value="Login successful" />
</c:redirect>

<%-- ── c:import — include content from URL ────────────────── --%>
<c:import url="/WEB-INF/jsp/header.jsp" />
<c:import url="https://example.com/rss" var="feed" charEncoding="UTF-8" />
```

TAG NOTES

- `<c:out value="${x}" />` — safest way to output any value; escapes by default
- `default` attr — rendered when value is null or empty; eliminates extra `<c:if>`
- `<c:url value="/path">` — prepends context path; appends JSESSIONID if needed
- `var` on c:url — stores URL string in scope for reuse; omit to output inline
- `<c:param>` — URL-encodes name and value; replaces manual `URLEncoder.encode()`
- `<c:redirect>` — calls response.sendRedirect(); execution stops after this tag
- `<c:import url="/internal">` — uses RequestDispatcher; `url="http://..."` fetches via HTTP

MODERN EQUIVALENT

- Thymeleaf `th:text` (escaped), `@{…}` URLs

PREFER / AVOID

- ✗ ~~`${x}` for output~~ → ✓ `<c:out value="${x}"/>`
- ✗ ~~hand-built URLs~~ → ✓ `<c:url>` + `<c:param>`

SECURITY CHECK

- `escapeXml="false"` turns XSS protection off
- `c:import` / `c:redirect` with user URLs = SSRF / open redirect

C:OUT ATTRIBUTES

ATTRIBUTE	REQUIRED	DESCRIPTION
`value`	✔	Expression to output
`default`	✖	Output if value is null/empty
`escapeXml`	✖	true (default) — escapes HTML entities

C:URL ATTRIBUTES

ATTRIBUTE	REQUIRED	DESCRIPTION
`value`	✔	Base URL (context-relative or absolute)
`var`	✖	Store result in this scoped variable
`scope`	✖	Scope for var; default page
`context`	✖	Cross-context path prefix

C:OUT VS BARE EL

SYNTAX	XSS SAFE?	NULL OUTPUT
`${user.bio}`	✖	""
`<c:out value="${user.bio}"/>`	✔	""
`<c:out value="${x}" default="N/A"/>`	✔	"N/A"
`${fn:escapeXml(user.bio)}`	✔	""

⚠ COMMON MISTAKES

- Outputting user input with bare `${param.x}` instead of `<c:out value="${param.x}"/>` — the bare form does not escape HTML and is an XSS vector
- Building URLs by concatenating context path manually: `${pageContext.request.contextPath}/path?id=${id}` — parameter values are not URL-encoded; use `<c:url>` with `<c:param>`
- Using `<c:import url="https://...">` for arbitrary external URLs without a timeout — a slow or unresponsive external server will block the entire page rendering thread

✓ TIPS

- Use `<c:out>` as the default for all data output in JSPs — only switch to bare `${}` for values you control (constants, formatted numbers)
- Use `<c:url var="u">` and store the URL in a variable when the same URL appears more than once on the page — avoids repeating the param list
- Use `<c:redirect>` instead of `<% response.sendRedirect(); %>` — it is concise and consistent with the no-scriptlet style

QUICK SUMMARY

- `<c:out value="${x}">` — HTML-escapes output; use for all user data
- `default` — fallback when value is null or empty
- `<c:url value="/path"><c:param>` — context-aware, encoded URL builder
- `<c:redirect url="/path">` — 302; stops execution
- `<c:import>` — include internal or external URL content

JSTL Formatting (fmt:)

NUMBERS, DATES, AND I18N

The `fmt:` library handles locale-aware formatting for numbers and dates, as well as internationalization (i18n) via resource bundles. `<fmt:formatNumber>` formats decimals, percentages, and currency according to locale rules. `<fmt:formatDate>` formats `java.util.Date` values. `<fmt:message>` looks up a key in a resource bundle for translated strings — the foundation of multi-language JSP applications.

KEY FACTS

- Locale is determined by `<fmt:setLocale>`, the session, or the browser's `Accept-Language` header — in that priority order
- `fmt:formatDate` accepts `java.util.Date` or `java.sql.Date` only — not Java 8+ `LocalDate`
- Resource bundles are `.properties` files on the classpath: `messages_en.properties`, `messages_fr.properties`
- `<fmt:setBundle>` sets the bundle globally for the page; `<fmt:bundle>` sets it for a block
- Time zone affects date output: set with `<fmt:timeZone>` or `<fmt:setTimeZone>`

FMT: TAG USAGE

```
<%@ taglib prefix="fmt" uri="jakarta.tags.fmt" %>

<%-- ── Number formatting ──────────────────────────────── --%>
<fmt:formatNumber value="${price}" type="currency" />
<%-- en_US: $1,234.56   fr_FR: 1 234,56 € --%>

<fmt:formatNumber value="${price}" type="currency"
                  currencyCode="EUR" />        <%-- force EUR --%>
<fmt:formatNumber value="${rate}"  type="percent" />  <%-- 75% --%>
<fmt:formatNumber value="${val}"   type="number"
                  minFractionDigits="2"
                  maxFractionDigits="2" />     <%-- 1,234.56 --%>
<fmt:formatNumber value="${val}"   pattern="#,##0.00" />

<%-- ── Date formatting ────────────────────────────────── --%>
<fmt:formatDate value="${order.createdAt}" type="date"
                dateStyle="long" />           <%-- January 5, 2025 --%>
<fmt:formatDate value="${order.createdAt}" type="both"
                dateStyle="short" timeStyle="short" />
<fmt:formatDate value="${order.createdAt}"
                pattern="yyyy-MM-dd HH:mm" /> <%-- 2025-01-05 14:30 --%>

<%-- ── Time zone ──────────────────────────────────────── --%>
<fmt:timeZone value="America/New_York">
  <fmt:formatDate value="${ts}" type="both" />
</fmt:timeZone>
<fmt:setTimeZone value="UTC" scope="session" />

<%-- ── Locale ─────────────────────────────────────────── --%>
<fmt:setLocale value="fr_FR" scope="session" />

<%-- ── i18n messages ──────────────────────────────────── --%>
<fmt:setBundle basename="com.example.messages" scope="session" />
<fmt:message key="welcome.title" />
<fmt:message key="greeting">
  <fmt:param value="${user.name}" />
</fmt:message>

<%-- Scoped bundle (block only) --%>
<fmt:bundle basename="com.example.messages">
  <h1><fmt:message key="page.title" /></h1>
  <p><fmt:message key="page.intro" /></p>
</fmt:bundle>
```

TAG NOTES

- → `type="currency"` — formats with locale's currency symbol and grouping
- → `currencyCode="EUR"` — override currency symbol using ISO 4217 code
- → `pattern` — Java `DecimalFormat` / `SimpleDateFormat` pattern string
- → `dateStyle / timeStyle` — short | medium | long | full; uses locale rules
- → `<fmt:message key="k">` — looks up key in active bundle; falls back to key name if not found
- → `<fmt:param value="${x}">` — substitutes into `{0}`, `{1}` placeholders in bundle string
- → `basename` — classpath base name without locale suffix or `.properties`

FMT:FORMATNUMBER TYPE VALUES

TYPE	OUTPUT EXAMPLE (EN_US)
`number`	1,234.56
`currency`	$1,234.56
`percent`	75%

FMT:FORMATDATE TYPE & DATESTYLE

TYPE	SHOWS
`date`	Date only
`time`	Time only
`both`	Date and time

DATESTYLE / TIMESTYLE	EXAMPLE (EN_US)
`short`	1/5/25
`medium`	Jan 5, 2025
`long`	January 5, 2025
`full`	Sunday, January 5, 2025

MODERN EQUIVALENT

- Thymeleaf `#temporals`, `#numbers` · `Intl` in JS

PREFER / AVOID

- ✗ ~~`SimpleDateFormat` in scriptlets~~ → ✓ `fmt:formatDate`
- ✗ ~~hard-coded strings~~ → ✓ `fmt:message` **bundles**

⚠ COMMON MISTAKES

- Passing a `LocalDate` or `LocalDateTime` to `<fmt:formatDate>` — it only accepts `java.util.Date`; convert first with `Date.from(localDate.atStartOfDay().toInstant(ZoneOffset.UTC))`
- Forgetting to set a bundle before using `<fmt:message>` — without a bundle, every key renders as the key name literally
- Using `type="currency"` without setting locale — defaults to the server JVM locale, which may not match the user's country

✓ TIPS

- Set locale and bundle in a shared fragment included at the top of every page — centralizes the i18n setup in one place
- Use `pattern="yyyy-MM-dd"` for machine-readable dates in hidden fields or data attributes — avoids locale-dependent parsing issues
- Store the resolved locale in session scope after detection — avoids re-checking `Accept-Language` on every request

QUICK SUMMARY

- `<fmt:formatNumber type="currency">` — locale-aware currency output
- `<fmt:formatDate pattern="yyyy-MM-dd">` — date formatting
- `<fmt:setLocale value="fr_FR">` — set locale for page/session
- `<fmt:setBundle basename="...">` — load .properties resource bundle
- `<fmt:message key="k">` — i18n string lookup with {0} params

JSTL Functions (fn:)

EL FUNCTION LIBRARY

The `fn:` library provides 16 string and collection utility functions callable directly inside EL expressions — `${fn:method(arg)}`. They are not tags; they return values rather than producing output. This sheet covers all 16 functions with practical examples. Declare the library with `<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>`. See Sheet 17 for custom EL function registration.

KEY FACTS

- `fn:` functions are called inside `${}`, not as tags — they return a value
- `fn:length()` works on String, array, Collection, and Map — returns int
- `fn:escapeXml()` escapes the 5 XML special chars: `< > & " '`
- `fn:split()` returns `String[]`; iterate with `<c:forEach>`
- `fn:join()` accepts only `String[]` — not List or Collection
- String indices in all fn: functions are zero-based

ALL FN: FUNCTIONS WITH EXAMPLES

```
<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>
<%@ taglib prefix="c"  uri="jakarta.tags.core" %>

<%-- Length --%>
${fn:length("hello")}                    <%-- 5 --%>
${fn:length(productList)}                <%-- list size --%>

<%-- Boolean tests --%>
${fn:contains("foobar", "foo")}          <%-- true --%>
${fn:containsIgnoreCase("Hello","hello")}<%-- true --%>
${fn:startsWith("index.jsp","index")}    <%-- true --%>
${fn:endsWith("logo.png",".png")}        <%-- true --%>
${fn:indexOf("foobar","bar")}            <%-- 3, -1 if absent --%>

<%-- Extraction --%>
${fn:substring("Hello World",6,11)}      <%-- "World" --%>
${fn:substringBefore("user@host","@")}   <%-- "user" --%>
${fn:substringAfter("user@host","@")}    <%-- "host" --%>

<%-- Transform --%>
${fn:toUpperCase("hello")}               <%-- "HELLO" --%>
${fn:toLowerCase("HELLO")}               <%-- "hello" --%>
${fn:trim("  hi  ")}                     <%-- "hi" --%>
${fn:replace("a-b-c","-","_")}           <%-- "a_b_c" --%>
${fn:escapeXml("<b>bold</b>")}           <%-- <b>bold</b> --%>

<%-- Split and Join --%>
<c:set var="parts" value="${fn:split('a,b,c',',')}" />
${parts[0]}  ${parts[1]}  ${parts[2]}    <%-- a  b  c --%>
${fn:join(parts," | ")}                  <%-- "a | b | c" --%>

<%-- Common patterns --%>
<c:if test="${fn:length(errors) gt 0}">
  <ul>
    <c:forEach items="${errors}" var="e">
      <li><c:out value="${e}"/></li>
    </c:forEach>
  </ul>
</c:if>

<%-- Safe truncation with ellipsis --%>
${fn:length(post.body) gt 200
    ? fn:substring(post.body,0,200) += '...'
    : post.body}
```

FUNCTION NOTES

- → `fn:contains(s,sub)` — case-sensitive; use `containsIgnoreCase` for flexible matching
- → `fn:indexOf(s,sub)` — returns -1 if not found; useful for guard checks
- → `fn:substring(s,begin,end)` — begin inclusive, end exclusive, 0-based
- → `fn:replace(s,before,after)` — replaces ALL occurrences, not just first
- → `fn:split(s,delim)` — delimiter is a single literal string (unlike c:forTokens)
- → `fn:join(arr,sep)` — accepts `String[]` only; result from fn:split works directly
- → `fn:escapeXml(s)` — essential before printing any user data in HTML context

COMPLETE FN: REFERENCE

FUNCTION	RETURNS
`fn:length(obj)`	int — String len or collection size
`fn:contains(s, sub)`	boolean — case-sensitive
`fn:containsIgnoreCase(s, sub)`	boolean
`fn:startsWith(s, prefix)`	boolean
`fn:endsWith(s, suffix)`	boolean
`fn:indexOf(s, sub)`	int — -1 if absent
`fn:substring(s, begin, end)`	String — 0-based, end exclusive
`fn:substringBefore(s, sub)`	String — before first match
`fn:substringAfter(s, sub)`	String — after first match
`fn:replace(s, before, after)`	String — all occurrences
`fn:split(s, delim)`	String[]
`fn:join(arr, sep)`	String — from String[] only
`fn:toUpperCase(s)`	String
`fn:toLowerCase(s)`	String
`fn:trim(s)`	String — strips whitespace
`fn:escapeXml(s)`	String — escapes < > & " '

FN: VS JAVA STRING EQUIVALENT

FN: CALL	JAVA EQUIVALENT
`fn:length(s)`	s.length()
`fn:contains(s,sub)`	s.contains(sub)
`fn:startsWith(s,p)`	s.startsWith(p)
`fn:indexOf(s,sub)`	s.indexOf(sub)
`fn:substring(s,b,e)`	s.substring(b,e)
`fn:replace(s,b,r)`	s.replace(b,r)
`fn:toUpperCase(s)`	s.toUpperCase()
`fn:trim(s)`	s.trim()
`fn:split(s,d)`	s.split(d) (regex)

MODERN EQUIVALENT

- Thymeleaf `#strings` · JS string methods

PREFER / AVOID

✗ ~~string logic in scriptlets~~ → ✓ `fn:` **functions**

⚠ COMMON MISTAKES

- Passing a `List` to `fn:join()` — it only accepts `String[]`; use `<c:forEach>` with manual separator logic for Lists
- Using `fn:split(s, ",")` and expecting regex behavior — the delimiter is a literal string, not a regex pattern; for regex splitting use Java in the servlet
- Skipping `fn:escapeXml()` for output that "looks safe" — a value like `AT&T` contains an ampersand that will break HTML if unescaped

✓ TIPS

- Combine functions: `${fn:toUpperCase(fn:trim(param.code))}` — fn: calls nest cleanly inside EL
- Guard against null before string functions: `${not empty s ? fn:toUpperCase(s) : ''}` — some functions throw on null input depending on EL version
- Use `fn:length()` with `gt` instead of `!= 0` for collection size checks — reads naturally: `${fn:length(list) gt 0}`

QUICK SUMMARY

- `${fn:length(x)}` — String length or collection size
- `${fn:escapeXml(s)}` — escape HTML entities; use on all output
- `${fn:contains / startsWith / endsWith / indexOf}` — boolean/int tests
- `${fn:split(s,d)}` → String[]; `${fn:join(arr,sep)}` → String
- fn: functions nest: `${fn:trim(fn:toLowerCase(s))}`

Tag Files

REUSABLE TEMPLATES WITHOUT JAVA

Tag files are the simplest way to create a custom tag — you write JSP syntax in a `.tag` file and place it under `WEB-INF/tags/`. No Java class, no TLD required. The container compiles the tag file into a `SimpleTag` handler automatically. Tag files are ideal for reusable layout components (cards, panels, form fields) that need named attributes and optional body content.

KEY FACTS

- Files go in `WEB-INF/tags/` (or subdirectories); extension is `.tag` or `.tagx`
- No TLD required — declare with `tagdir="/WEB-INF/tags/"` in the taglib directive
- `<%@ attribute name="title" required="true" %>` declares an attribute the caller must or may supply
- `<jsp:doBody />` renders the body content provided by the calling page
- Tag file implicit objects: `request`, `response`, `session`, `application`, `out`, `jspContext`

TAG FILE CREATION & USAGE

```
<%-- ── WEB-INF/tags/panel.tag ──────────────────────────── --%>
<%@ tag description="Renders a content panel" pageEncoding="UTF-8" %>
<%@ attribute name="title"    required="true"  rtexprvalue="true" %>
<%@ attribute name="cssClass" required="false" rtexprvalue="true" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<div class="panel ${empty cssClass ? '' : cssClass}">
  <div class="panel-header">
    <h2>${title}</h2>
  </div>
  <div class="panel-body">
    <jsp:doBody />        <%-- renders caller's body content --%>
  </div>
</div>

<%-- ── WEB-INF/tags/ui/formField.tag ──────────────────── --%>
<%@ tag pageEncoding="UTF-8" %>
<%@ attribute name="label"    required="true" %>
<%@ attribute name="name"     required="true" %>
<%@ attribute name="value"    required="false" %>
<%@ attribute name="type"     required="false" %>
<c:set var="inputType" value="${empty type ? 'text' : type}" />
<div class="form-group">
  <label for="${name}">${label}</label>
  <input type="${inputType}" id="${name}" name="${name}"
         value="<c:out value='${value}'/>" class="form-control">
</div>

<%-- ── Calling page: declare and use tag files ──────────── --%>
<%@ taglib prefix="ui"  tagdir="/WEB-INF/tags"    %>
<%@ taglib prefix="frm" tagdir="/WEB-INF/tags/ui" %>

<ui:panel title="User Profile" cssClass="primary">
  <p>Account details for ${user.name}</p>
  <frm:formField label="Email"    name="email"    value="${user.email}" />
  <frm:formField label="Username" name="username" value="${user.username}" />
</ui:panel>
```

DIRECTIVE NOTES

- → `<%@ tag %>` — tag file equivalent of `<%@ page %>`; sets encoding, description, etc.
- → `<%@ attribute name="x" required="true" %>` — declares a named attribute; available as EL var `${x}`
- → `rtexprvalue="true"` — allows EL expressions as attribute values (default true)
- → `<jsp:doBody />` — inserts caller's body; omit if tag has no body
- → `tagdir="/WEB-INF/tags"` — declares a tag file library; each `.tag` file becomes a tag
- → Subdirectories — files in `WEB-INF/tags/ui/` need a separate taglib with `tagdir="/WEB-INF/tags/ui"`
- → `<%@ variable %>` — exposes a scoped variable to the calling page (advanced usage)

MODERN EQUIVALENT

- Thymeleaf fragments with parameters · React components

PREFER / AVOID

- ✗ ~~copy-pasted markup~~ → ✓ **a `.tag` file**
- ✗ ~~classic handlers for simple markup~~ → ✓ **tag files**

TAG FILE DIRECTIVES

DIRECTIVE	PURPOSE
`<%@ tag %>`	Page-like config for the tag file
`<%@ attribute %>`	Declare a named attribute
`<%@ variable %>`	Export a variable to caller scope
`<%@ taglib %>`	Import other tag libraries
`<%@ include %>`	Static include inside tag file

<%@ ATTRIBUTE %> PROPERTIES

ATTRIBUTE	DESCRIPTION
`name`	Attribute name (required)
`required`	true / false; default false
`rtexprvalue`	Allow EL; default true
`type`	Java type; default String
`description`	Documentation string
`fragment`	true = JspFragment body attribute

TAG FILE VS CLASSIC TAG HANDLER

FEATURE	TAG FILE	CLASSIC HANDLER
`Language`	JSP syntax	Java class
`TLD required`	✖	✔
`Compile step`	Auto	Manual
`Best for`	Layout, markup	Logic-heavy tags
`JSP/EL access`	✔	Via PageContext

⚠ COMMON MISTAKES

- Placing tag files at the context root instead of `WEB-INF/tags/` — they must be inside WEB-INF to be treated as tag files; outside WEB-INF they are accessible as plain JSPs
- Using `uri=` in the taglib directive for a tag file library instead of `tagdir=` — tag files don't have a TLD URI; they use directory-based declaration
- Forgetting `<jsp:doBody />` in a tag that expects body content — without it, the caller's body is silently discarded

✓ TIPS

- Use tag files for any HTML structure that appears more than twice — panels, form groups, nav items, table rows — they are far easier to maintain than repeated markup
- Organize tag files into subdirectories by component type (`WEB-INF/tags/layout/` , `WEB-INF/tags/form/`) and declare each directory as a separate prefix
- Use `<c:out value='${attr}'/>` inside tag files to output attribute values — attribute values may contain user data and should be escaped

QUICK SUMMARY

- `WEB-INF/tags/name.tag` — tag file; no Java, no TLD needed
- `<%@ attribute name="x" required="true">` — declare attribute
- `<jsp:doBody />` — render caller's body content
- `tagdir="/WEB-INF/tags"` — declare in taglib directive (not uri=)
- Best for — markup-heavy reusable UI components

Classic Tag Handlers

TagSupport · BodyTagSupport · doStartTag · doEndTag · SKIP_BODY · EVAL_BODY_INCLUDE · doAfterBody

THE CLASSIC TAG API

Classic tag handlers implement `Tag` or `BodyTag` interfaces — usually by extending `TagSupport` or `BodyTagSupport`. They are stateful Java objects, reused across invocations in a pool. The lifecycle is: `doStartTag()` → (optionally evaluate body) → `doEndTag()`. The return value of each method controls what happens next. `BodyTagSupport` adds `doAfterBody()` for tags that loop over their body.

TAGSUPPORT & BODYTAGSUPPORT EXAMPLES

```
// ── Simple tag: outputs a greeting ──────────────────────
public class GreetTag extends TagSupport {
    private String name;

    public void setName(String name) { this.name = name; }

    @Override
    public int doStartTag() throws JspException {
        try {
            pageContext.getOut().print("Hello, " + name + "!");
        } catch (IOException e) {
            throw new JspException(e);
        }
        return SKIP_BODY;          // no body to render
    }

    @Override
    public void release() {
        super.release();
        name = null;               // reset pooled state
    }
}

// ── Body tag: repeats body N times ──────────────────────
public class RepeatTag extends BodyTagSupport {
    private int times;
    private int count;

    public void setTimes(int times) { this.times = times; }

    @Override
    public int doStartTag() {
        count = 0;
        return times > 0 ? EVAL_BODY_BUFFERED : SKIP_BODY;
    }

    @Override
    public int doAfterBody() throws JspException {
        count++;
        if (count < times) {
            return EVAL_BODY_AGAIN; // repeat
        }
        try {
            getBodyContent().writeOut(getPreviousOut());
        } catch (IOException e) { throw new JspException(e); }
        return SKIP_BODY;          // done
    }
}
```

LIFECYCLE NOTES

- → `setPageContext(pc)` — called first; stores PageContext (inherited from TagSupport)
- → `setParent(tag)` — called if tag is nested; stores parent tag reference
- → `setXxx(val)` — one setter per declared attribute; called before doStartTag()
- → `doStartTag()` — return SKIP_BODY or EVAL_BODY_INCLUDE/BUFFERED
- → `doAfterBody()` — BodyTagSupport only; EVAL_BODY_AGAIN loops body
- → `doEndTag()` — return EVAL_PAGE to continue or SKIP_PAGE to abort
- → `release()` — called when tag is returned to pool; nullify all fields
- → `getBodyContent()` — returns buffered body as BodyContent for inspection/modification

KEY FACTS

- Classic tags are pooled and reused — always implement `release()` to reset state
- `doStartTag()` returns: `SKIP_BODY` (skip body), `EVAL_BODY_INCLUDE` (render body once)
- `BodyTagSupport.doStartTag()` can also return `EVAL_BODY_BUFFERED` (buffer body for inspection)
- `doEndTag()` returns: `SKIP_PAGE` (abort rest of JSP), `EVAL_PAGE` (continue)
- `doAfterBody()` returns: `EVAL_BODY_AGAIN` (repeat body) or `SKIP_BODY` (stop)
- Access page context via the inherited `pageContext` field

RETURN VALUE CONSTANTS

METHOD	CONSTANT	EFFECT
`doStartTag()`	SKIP_BODY	Skip body entirely
`doStartTag()`	EVAL_BODY_INCLUDE	Render body to response
`doStartTag()`	EVAL_BODY_BUFFERED	Buffer body (BodyTag only)
`doAfterBody()`	SKIP_BODY	Stop iterating body
`doAfterBody()`	EVAL_BODY_AGAIN	Repeat body evaluation
`doEndTag()`	EVAL_PAGE	Continue page rendering
`doEndTag()`	SKIP_PAGE	Abort rest of JSP page

CLASSIC VS SIMPLETAG API

FEATURE	CLASSIC	SIMPLETAG
`Base class`	TagSupport / BodyTagSupport	SimpleTagSupport
`Pooled`	✔	✖ (new per call)
`Body iteration`	EVAL_BODY_AGAIN	invoke() in loop
`Complexity`	Higher	Lower
`Preferred for`	Legacy; performance loops	New development

PREFER / AVOID

- ✗ ~~`TagSupport` for new tags~~ → ✓ `SimpleTagSupport` **or tag files**
- ✗ ~~state in tag fields across uses~~ → ✓ **reset in** `release()` **/ use SimpleTag**

MODERN EQUIVALENT

- Thymeleaf custom dialects / processors

⚠ COMMON MISTAKES

- Not implementing `release()` — classic tags are pooled; instance fields from a previous invocation bleed into the next call if not reset
- Returning `EVAL_BODY_BUFFERED` without reading from `getBodyContent()` — the buffered body is silently discarded; you must call `writeOut(getPreviousOut())` to emit it
- Calling `pageContext.getOut().print()` in `doAfterBody()` while using EVAL_BODY_BUFFERED — write to `getPreviousOut()` instead; `getOut()` returns the body buffer, not the response

✓ TIPS

- Prefer `SimpleTagSupport` for all new custom tags — the API is simpler, not pooled (no release() pitfall), and handles body content more cleanly
- Use classic `BodyTagSupport` when you need to inspect or transform the body content before emitting it (e.g., a Markdown-rendering tag)
- Always wrap `pageContext.getOut().print()` in a try/catch for `IOException` and rethrow as `JspException` — the tag interface requires it

QUICK SUMMARY

- `TagSupport` — doStartTag() + doEndTag(); no body manipulation
- `BodyTagSupport` — adds doAfterBody(); can buffer and transform body
- `SKIP_BODY / EVAL_BODY_INCLUDE` — doStartTag() return values
- `EVAL_BODY_AGAIN` — repeat body; used for iteration tags
- `release()` — must reset all fields; tags are pooled

SimpleTag Handlers

THE MODERN TAG API

`SimpleTagSupport` is the preferred API for new custom tags. Unlike classic tags, a new instance is created for every tag invocation — no pooling, no `release()` required. The entire logic lives in a single `doTag()` method. Body content is accessed via `getJspBody().invoke(null)` to render it, or `invoke(writer)` to capture it. Implement `DynamicAttributes` to accept arbitrary attributes without declaring each one in the TLD.

KEY FACTS

- New instance per invocation — no pooling, no state bleed between calls
- Single method: `doTag()` — no start/end/afterBody split
- `getJspBody()` returns a `JspFragment`; call `invoke(null)` to render it to response
- `invoke(writer)` captures body into a `StringWriter` for inspection/modification
- Throw `SkipPageException` from `doTag()` to stop rendering the rest of the JSP
- Access web context via `getJspContext()` — cast to `PageContext` for HTTP-specific methods

SIMPLETAGSUPPORT KEY METHODS

METHOD	DESCRIPTION
`doTag()`	Main lifecycle method; implement this
`getJspBody()`	Returns body as JspFragment; null if empty
`getJspContext()`	JspContext / PageContext
`getParent()`	Enclosing tag instance, if nested
`findAncestorWithClass(tag, cls)`	Walk up tag stack to find ancestor

JSPFRAGMENT METHODS

METHOD	EFFECT
`invoke(null)`	Render body to current response output
`invoke(writer)`	Capture body into the supplied Writer
`getJspContext()`	JspContext of the invoking page

PATTERN	CODE
`Render body`	getJspBody().invoke(null)
`Capture body`	StringWriter sw = new StringWriter(); getJspBody().invoke(sw);
`Repeat body`	Loop calling invoke(null) N times
`Stop page`	throw new SkipPageException()

API NOTES

- → `doTag()` — single lifecycle method; replaces doStartTag/doEndTag/doAfterBody
- → `getJspBody()` — returns `JspFragment`; null if tag was used without body
- → `jspBody.invoke(null)` — render body to the response output stream
- → `jspBody.invoke(writer)` — capture body into a Writer for transformation
- → `getJspContext()` — returns `JspContext`; cast to `PageContext` for HTTP access
- → `getJspContext().getOut()` — JspWriter for writing to response
- → `DynamicAttributes.setDynamicAttribute(uri, name, val)` — called for each unknown attribute; requires TLD `dynamic-attributes="true"`
- → `throw new SkipPageException()` — stops rest of JSP; equivalent to classic SKIP_PAGE

SIMPLETAGSUPPORT EXAMPLES

```
// ── Basic SimpleTag: conditional render ──────────────
public class IfRoleTag extends SimpleTagSupport {
    private String role;
    public void setRole(String role) { this.role = role; }

    @Override
    public void doTag() throws JspException, IOException {
        PageContext pc = (PageContext) getJspContext();
        HttpSession session = pc.getSession();
        User user = (User) session.getAttribute("currentUser");

        if (user != null && role.equals(user.getRole())) {
            getJspBody().invoke(null); // render body to response
        }
        // if condition fails, body is not rendered
    }
}

// ── Tag with body capture and transform ──────────────
public class MarkdownTag extends SimpleTagSupport {
    @Override
    public void doTag() throws JspException, IOException {
        StringWriter sw = new StringWriter();
        getJspBody().invoke(sw);        // capture body to string
        String markdown = sw.toString();
        String html = MarkdownParser.toHtml(markdown);
        getJspContext().getOut().print(html);
    }
}

// ── DynamicAttributes: accept any attribute ──────────
public class HtmlTag extends SimpleTagSupport
        implements DynamicAttributes {
    private final Map<String,Object> attrs = new LinkedHashMap<>();
    private String tag;
    public void setTag(String tag) { this.tag = tag; }

    @Override
    public void setDynamicAttribute(String uri, String name,
            Object value) {
        attrs.put(name, value);
    }

    @Override
    public void doTag() throws JspException, IOException {
        JspWriter out = getJspContext().getOut();
        out.print("<" + tag);
        for (Map.Entry<String,Object> e : attrs.entrySet())
            out.print(" " + e.getKey() + "=\"" + e.getValue() + "\"");
        out.print(">");
        if (getJspBody() != null) getJspBody().invoke(null);
        out.print("</" + tag + ">");
    }
}
```

PREFER / AVOID

✗ ~~`BodyTagSupport`~~ ~~iteration~~ → ✓ **`JspFragment.invoke()` in a loop**

MODERN EQUIVALENT

- Thymeleaf element / attribute processors

⚠ COMMON MISTAKES

- Calling `getJspBody()` when the tag was used as a self-closing tag — returns null; always null-check before invoking the body
- Casting `getJspContext()` to `PageContext` in environments where it is not an HTTP context — add a null/instanceof check when portability matters
- Forgetting to add `<dynamic-attributes>true</dynamic-attributes>` to the TLD when implementing `DynamicAttributes` — without it, unknown attributes cause a translation error instead of calling `setDynamicAttribute()`

✓ TIPS

- Use `SimpleTagSupport` for all new tags — the one-method API is much easier to reason about than the classic multi-method lifecycle
- Implement `DynamicAttributes` for HTML-passthrough tags — lets callers add arbitrary HTML attributes (class, data-*, aria-*) without declaring each one in the TLD
- For tags that repeat their body, call `getJspBody().invoke(null)` in a Java loop — cleaner than the classic `EVAL_BODY_AGAIN` pattern

QUICK SUMMARY

- `SimpleTagSupport.doTag()` — single method; new instance per call
- `getJspBody().invoke(null)` — render body to response
- `getJspBody().invoke(writer)` — capture body for transformation
- `DynamicAttributes` — accept arbitrary tag attributes
- `throw new SkipPageException()` — stop rest of JSP page

Tag Library Descriptors (TLD)

.tld file · <uri> · <tag> · <attribute> · <function> · META-INF · body-content · dynamic-attributes

DESCRIBING A TAG LIBRARY

A **TLD** (Tag Library Descriptor) is an XML file that maps a logical URI to a set of tag handler classes and EL functions. The container reads TLDs at startup from `WEB-INF/*.tld` or from `META-INF/*.tld` inside JARs in `WEB-INF/lib/`. Each tag's attributes, body content type, and handler class are declared here. Tag files (`.tag`) do not need a TLD — the container generates the metadata automatically.

KEY FACTS

- TLD namespace: `https://jakarta.ee/xml/ns/jakartaee` (Jakarta EE 9+); older: `http://java.sun.com/xml/ns/javaee`
- Place in `WEB-INF/` for app-local libraries; in `META-INF/` inside a JAR for distributable libraries
- `<body-content>`: `empty` | `JSP` | `scriptless` | `tagdependent`
- `rtexprvalue` on attribute — true allows EL; false requires literal string
- For SimpleTag / tag files, use `scriptless` for body-content (no scriptlets allowed in body)

TLD ELEMENT NOTES

- → `<uri>` — logical identifier used in `taglib` directive `uri=`; need not be a real URL
- → `<tag-class>` — fully qualified class implementing Tag, SimpleTag, etc.
- → `<body-content>empty</body-content>` — tag must not have body content
- → `<body-content>scriptless</body-content>` — body allows EL and tags but not scriptlets; required for SimpleTag
- → `<body-content>JSP</body-content>` — body allows all JSP syntax including scriptlets; classic tags only
- → `<body-content>tagdependent</body-content>` — body is raw text; container does not process JSP/EL in it
- → `<dynamic-attributes>true</dynamic-attributes>` — tag class must implement DynamicAttributes

MODERN EQUIVALENT

- No descriptors in Thymeleaf — dialect classes register themselves

PREFER / AVOID

- ✗ ~~TLDs for tag files~~ → ✓ `tagdir` **(no TLD needed)**
- ✗ ~~`javax`~~ ~~TLD schema~~ → ✓ **Jakarta** `web-jsptaglibrary_3_0`

TLD STRUCTURE

```
/* WEB-INF/myapp.tld */
```

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib xmlns="https://jakarta.ee/xml/ns/jakartaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
            https://jakarta.ee/xml/ns/jakartaee/web-jsptaglibrary_3_0.xsd"
        version="3.0">

  <tlib-version>1.0</tlib-version>
  <short-name>myapp</short-name>
  <uri>https://myapp.example.com/tags</uri>

  <!-- ── Simple tag declaration ──────────────────── -->
  <tag>
    <name>ifRole</name>
    <tag-class>com.example.tags.IfRoleTag</tag-class>
    <body-content>scriptless</body-content>
    <attribute>
      <name>role</name>
      <required>true</required>
      <rtexprvalue>true</rtexprvalue>
      <type>java.lang.String</type>
    </attribute>
  </tag>

  <!-- ── Tag with dynamic attributes ─────────────── -->
  <tag>
    <name>html</name>
    <tag-class>com.example.tags.HtmlTag</tag-class>
    <body-content>scriptless</body-content>
    <dynamic-attributes>true</dynamic-attributes>
    <attribute>
      <name>tag</name>
      <required>true</required>
      <rtexprvalue>true</rtexprvalue>
    </attribute>
  </tag>

  <!-- ── EL function declaration ─────────────────── -->
  <function>
    <name>formatCurrency</name>
    <function-class>com.example.ELFunctions</function-class>
    <function-signature>
      java.lang.String formatCurrency(java.math.BigDecimal, java.lang.String)
    </function-signature>
  </function>

</taglib>
```

BODY-CONTENT VALUES

VALUE	BODY ALLOWS	TYPICAL USE
`empty`	Nothing	Self-closing tags
`scriptless`	EL, tags (no scriptlets)	SimpleTag, tag files
`JSP`	All JSP syntax	Classic tags
`tagdependent`	Raw text only	SQL, template content

TLD DISCOVERY LOCATIONS

LOCATION	WHEN USED
`WEB-INF/*.tld`	App-local tag library
`WEB-INF/tlds/*.tld`	App-local (subdirectory)
`META-INF/*.tld (in JAR)`	Distributable library JAR
`Explicit uri= path`	e.g. uri="/WEB-INF/myapp.tld"

<ATTRIBUTE> CHILD ELEMENTS

ELEMENT	DESCRIPTION
`<name>`	Attribute name (required)
`<required>`	true / false
`<rtexprvalue>`	true = EL allowed
`<type>`	Java type (default String)
`<fragment>`	true = JspFragment attribute
`<description>`	Documentation

⚠ COMMON MISTAKES

- Using `<body-content>JSP</body-content>` for a SimpleTag — SimpleTag implementations cannot handle scriptlet body content; use `scriptless`
- Mismatching the `<uri>` in the TLD with the `uri=` in the taglib directive — they must be identical strings; the container won't find the TLD
- Forgetting `<dynamic-attributes>true</dynamic-attributes>` when the handler implements `DynamicAttributes` — unknown attributes will cause a translation error instead of being passed to the handler

✓ TIPS

- Package distributable tag libraries as a JAR with the TLD in `META-INF/` — consumers just add the JAR to `WEB-INF/lib/` and the container discovers the TLD automatically
- Use a short-form `<uri>` like `myapp.tags` rather than a full URL — it's easier to type in taglib directives and doesn't imply a real HTTP resource
- Validate your TLD against the Jakarta schema during build — malformed TLDs produce cryptic "cannot find taglib" errors at deploy time

QUICK SUMMARY

- `<uri>` — logical name matching taglib directive uri=
- `<tag-class>` — fully qualified handler class
- `<body-content>scriptless</body-content>` — use for SimpleTag / tag files
- `<dynamic-attributes>true</dynamic-attributes>` — enable DynamicAttributes
- META-INF/*.tld in JAR — auto-discovered by container

Servlet-to-JSP Data Flow

THE MVC PATTERN WITH SERVLETS & JSP

In the standard JSP/Servlet MVC pattern, the **servlet is the controller**: it receives requests, invokes service/DAO classes, and places results into request (or session) attributes. It then `forward()`s to a **JSP view**, which reads the attributes via EL and renders HTML. The JSP never talks to services or databases. This clean separation makes the application testable and maintainable.

KEY FACTS

- `RequestDispatcher.forward()` — same request/response; attributes survive; URL in browser does not change
- `response.sendRedirect()` — new request; attributes lost; URL changes; use for Post/Redirect/Get
- Put all data the JSP needs into `request.setAttribute()` before forwarding
- Post/Redirect/Get (PRG): on successful POST, redirect to a GET — prevents double-submission on refresh
- `@WebServlet("/path")` annotation replaces `<servlet>` / `<servlet-mapping>` in web.xml

FORWARD() VS SENDREDIRECT()

FEATURE	FORWARD()	SENDREDIRECT()
HTTP request	Same request	New request (302 → GET)
URL in browser	Unchanged	Changes to new URL
Request attrs	✔ survive	✖ lost
Session attrs	✔	✔
Use for	GET → JSP view	After POST (PRG)

MVC ROLE DIVISION

LAYER	CLASS	RESPONSIBILITY
Controller	HttpServlet	Route, validate, call services, set attrs, forward
Model	POJO / Service / DAO	Business logic, data access
View	JSP + JSTL/EL	Read attrs, render HTML — no logic

PATTERN NOTES

- → `req.setAttribute("key", val)` — store any Java object; JSP reads via EL `${key}`
- → `getRequestDispatcher("/WEB-INF/jsp/x.jsp").forward(req,res)` — delegate to JSP; URL stays the same
- → Forward vs redirect — forward shares request attributes; redirect loses them (use session for flash messages)
- → PRG — POST saves data, redirect to GET page; refresh re-issues safe GET, not the POST
- → `@WebServlet("/path")` — annotation-based URL mapping; no web.xml entry needed
- → JSP view responsibility — only EL reads, JSTL tags, zero business logic

PREFER / AVOID

- ✗ ~~POST that renders a page~~ → **✓ Post/Redirect/Get**
- ✗ ~~servlet per page~~ → **✓ one controller per feature**

SECURITY CHECK

- Forms need CSRF tokens (not built into JSP)
- Spring Security adds them automatically

SERVLET CONTROLLER + JSP VIEW PATTERN

```
// ── Servlet controller ──────────────────────────────
@WebServlet("/products")
public class ProductServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req,
                         HttpServletResponse res)
            throws ServletException, IOException {

        // 1. Parse / validate input
        String cat = req.getParameter("category");

        // 2. Call service layer
        List<Product> products = productService.findByCategory(cat);

        // 3. Place data into request scope
        req.setAttribute("products", products);
        req.setAttribute("category", cat);
        req.setAttribute("pageTitle", cat + " Products");

        // 4. Forward to JSP view
        req.getRequestDispatcher("/WEB-INF/jsp/products.jsp")
           .forward(req, res);
    }

    @Override
    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
            throws ServletException, IOException {
        // Process POST (form submit)
        productService.save(buildFromRequest(req));

        // Post/Redirect/Get — prevent double-submit
        res.sendRedirect(req.getContextPath() + "/products");
    }
}
```

```
<%-- ── JSP view: /WEB-INF/jsp/products.jsp ──────────── --%>
<%@ taglib prefix="c"   uri="jakarta.tags.core"      %>
<%@ taglib prefix="fn"  uri="jakarta.tags.functions" %>
<%@ page pageEncoding="UTF-8" %>
<!doctype html>
<html><head><title>${pageTitle}</title></head>
<body>
<h1>${fn:escapeXml(category)} Products</h1>
<p>${fn:length(products)} items found</p>
<ul>
<c:forEach items="${products}" var="p">
  <li><c:out value="${p.name}"/> — ${p.price}</li>
</c:forEach>
</ul>
</body></html>
```

MODERN EQUIVALENT

- Spring MVC: `@Controller` + `Model` + view name
- Next.js: Server Components fetch, then render

⚠ COMMON MISTAKES

- Forwarding after a POST without a redirect — user hitting refresh re-submits the POST form, duplicating the action (duplicate order, duplicate record)
- Calling `forward()` after any output has been written to the response — once the buffer is flushed, forward throws `IllegalStateException`
- Accessing the database or calling services from inside the JSP using scriptlets — puts business logic in the view, making it untestable and hard to maintain

✓ TIPS

- Always forward to JSPs under `WEB-INF/jsp/` — they cannot be requested directly; every access goes through the servlet controller
- For flash messages after a PRG redirect, store the message in session, read and remove it in the next GET request — prevents the message showing again on refresh
- Use a Front Controller servlet (or a framework like Spring MVC) to centralize routing rather than one servlet per URL — reduces duplication of auth checks and error handling

QUICK SUMMARY

- Servlet = controller: call services, `setAttribute()`, then `forward()`
- JSP = view: read `${attrs}`, render HTML, zero business logic
- `forward()` — same request; attrs survive; URL unchanged
- `sendRedirect()` — new request; use after POST (PRG pattern)
- Store JSPs in `WEB-INF/jsp/` — controller-only access

Error Handling & Security Output

errorPage · isErrorPage · <error-page> · exception implicit · XSS · c:out · fn:escapeXml · CSP

ERROR PAGES & SECURE OUTPUT

JSP provides two mechanisms for error handling: per-page `errorPage` directives and global `<error-page>` entries in `web.xml`. The web.xml approach is strongly preferred — it covers all exceptions and HTTP status codes from one place. On the security side, every piece of data rendered in a JSP that originated from user input or a database must be HTML-escaped to prevent cross-site scripting (XSS). Use `<c:out>` or `fn:escapeXml()` for all such output.

KEY FACTS

- `<error-page>` in web.xml handles all JSPs and servlets — the per-page `errorPage` attribute only covers that one JSP
- The `exception` implicit object is only available when `<%@ page isErrorPage="true" %>` is set
- Bare `${}` EL does NOT HTML-escape — always use `<c:out>` for user-supplied or database data
- XSS via EL: `${param.name}` renders `<script>` tags literally if not escaped
- Set `Content-Security-Policy` response header to restrict script sources — defence-in-depth alongside output escaping

ERROR PAGES & SECURE OUTPUT PATTERNS

```
<!-- web.xml — global error page declarations -->
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/WEB-INF/jsp/error.jsp</location>
</error-page>
<error-page>
  <error-code>404</error-code>
  <location>/WEB-INF/jsp/404.jsp</location>
</error-page>
<error-page>
  <error-code>403</error-code>
  <location>/WEB-INF/jsp/403.jsp</location>
</error-page>
```

```
<%-- ── error.jsp — the error page itself ──────────────────── --%>
<%@ page isErrorPage="true" pageEncoding="UTF-8"
         contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<!doctype html>
<html><head><title>Error</title></head><body>
<h1>Something went wrong</h1>
<%-- exception implicit object available here --%>
<p>Reference: ${pageContext.errorData.requestURI}</p>
<% if (exception != null) {
       application.log("Unhandled error", exception); } %>
</body></html>

<%-- ── Per-page error page (less preferred) ──────────────── --%>
<%@ page errorPage="/WEB-INF/jsp/error.jsp" %>

<%-- ── SECURE OUTPUT — always escape user data ──────────── --%>
<%@ taglib prefix="c"  uri="jakarta.tags.core"      %>
<%@ taglib prefix="fn" uri="jakarta.tags.functions" %>

<%-- WRONG — XSS vulnerable --%>
Hello, ${param.name}!

<%-- RIGHT — use c:out (escapes by default) --%>
Hello, <c:out value="${param.name}" />!

<%-- RIGHT — use fn:escapeXml in EL context --%>
<input value="${fn:escapeXml(param.name)}">

<%-- Set CSP header to restrict script sources --%>
<% response.setHeader("Content-Security-Policy",
       "default-src 'self'; script-src 'self'"); %>

<%-- HTTP-only, Secure cookies --%>
<% Cookie c = new Cookie("sessionId", token);
   c.setHttpOnly(true);
   c.setSecure(true);
   response.addCookie(c); %>
```

NOTES

- → `<error-page><exception-type>` — catches that exception type and all subclasses
- → `<error-page><error-code>404</error-code>` — catches HTTP status 404 from any servlet or JSP
- → `isErrorPage="true"` — required to access `exception` implicit object
- → `pageContext.errorData` — available on web.xml error pages; has requestURI, statusCode, throwable
- → `<c:out value="${x}">` — escapes `< > & " '` ; safest default for all output
- → `fn:escapeXml(s)` — use inside EL attribute values where c:out can't be placed
- → `HttpOnly + Secure` cookie flags — prevent JS access to session cookie; force HTTPS

WEB.XML ERROR PAGE VS PAGE DIRECTIVE

FEATURE	WEB.XML <ERROR-PAGE>	ERRORPAGE DIRECTIVE
Coverage	All servlets & JSPs	That JSP only
HTTP status codes	✔	✖
exception types	✔	✔
errorData object	✔	✖
Preferred	✔	✖

OUTPUT ESCAPING — WHEN TO USE WHAT

CONTEXT	USE
HTML text node	<c:out value="${x}"/>
HTML attribute	${fn:escapeXml(x)} or c:out
URL parameter	<c:url> + <c:param>
Trusted HTML (CMS)	<c:out escapeXml="false">
JSON in script tag	Server-side JSON encoder

SECURITY HEADERS & COOKIE FLAGS

HEADER / FLAG	PURPOSE
Content-Security-Policy	Restrict script/style/image sources
X-Content-Type-Options	nosniff — prevent MIME sniffing
X-Frame-Options	DENY / SAMEORIGIN — block clickjacking
Strict-Transport-Security	Force HTTPS (HSTS)
Cookie: HttpOnly	Prevent JS access to cookie
Cookie: Secure	Send over HTTPS only
Cookie: SameSite=Strict	Block cross-site cookie sending

MODERN EQUIVALENT

- Spring `@ControllerAdvice` + Thymeleaf error templates

PREFER / AVOID

- ✗ ~~`exception.printStackTrace(out)`~~ → ✓ **log it; show a generic page**
- ✗ ~~per-page `errorPage`~~ → ✓ `<error-page>` **in** `web.xml`

⚠ COMMON MISTAKES

- Outputting `${param.x}` directly — EL does not escape HTML; any value containing `<script>` is rendered as executable JavaScript in the browser
- Relying on per-page `errorPage` for all error handling — exceptions from servlets, filters, and other JSPs bypass it; use `<error-page>` in web.xml
- Printing the exception stack trace to the response on production systems — exposes internal class names, library versions, and file paths that attackers can use for targeted exploits

✓ TIPS

- Make `<c:out>` the default for every data value in JSPs — only omit it for values you generate yourself (counters, formatted numbers, constants)
- Log exceptions server-side with `application.log("msg", exception)` or a proper logging framework; show only a friendly message and a reference code to the user
- Set security headers (CSP, X-Frame-Options, HSTS) in a servlet `Filter` applied to all responses — centralizes security policy in one place

QUICK SUMMARY

- `<error-page>` in web.xml — global; covers exceptions and HTTP codes
- `isErrorPage="true"` — enables `exception` implicit object
- `<c:out value="${x}">` — HTML-escape all user/DB data; default habit
- `fn:escapeXml(x)` — use inside EL attribute contexts
- HttpOnly + Secure cookie flags — protect session cookie from JS and HTTP

Sheet 33

Quick Reference — Syntax, Objects & EL

directives · scripting elements · implicit objects · scopes · EL syntax & implicit objects · EL operators · standard actions

DIRECTIVES

SYNTAX	DOES	#
`<%@ page import="java.util.*" %>`	imports	03
`<%@ page contentType="text/html;charset=UTF-8" %>`	response type	03
`<%@ page errorPage="err.jsp" %>`	error page	03
`<%@ page isELIgnored="false" %>`	EL on (default)	03
`<%@ include file="h.jspf" %>`	static include	05
`<%@ taglib prefix="c" uri="…" %>`	tag library	06

SCRIPTING (LEGACY)

SYNTAX	DOES	#
`<% code %>`	scriptlet	04
`<%= expr %>`	expression, no escaping	04
`<%! decl %>`	field / method	04
`<%-- note --%>`	JSP comment, not sent	04
`\${ … }`	escape EL literally	13
`Prefer EL + JSTL`	scriptlets are discouraged	04

IMPLICIT OBJECTS

OBJECT	TYPE	#
`request`	`HttpServletRequest`	07
`response`	`HttpServletResponse`	07
`session`	`HttpSession`	08
`application`	`ServletContext`	08
`out`	`JspWriter`	09
`pageContext`	`PageContext`	09
`exception`	error pages only	09

SCOPES

SCOPE	LIVES FOR	#
`page`	one page execution	10
`request`	one request (incl. forwards)	10
`session`	one user session	11
`application`	whole web app	11
`findAttribute(n)`	page → request → session → app	12
`setAttribute(n, v, SCOPE)`	store in a scope	12

EL SYNTAX

EXPRESSION	DOES	#
`${user.name}`	bean property	13
`${map["key"]}`	map entry	16
`${list[0]}`	list index	16
`${empty cart}`	null or empty	15
`${a ? b : c}`	conditional	15
`#{bean.prop}`	deferred (JSF)	13

EL IMPLICIT OBJECTS

OBJECT	HOLDS	#
`param / paramValues`	request parameters	14
`header / headerValues`	request headers	14
`cookie`	cookies by name	14
`initParam`	context init params	14
`requestScope / sessionScope`	scope maps	14
`pageContext.request.contextPath`	app path	14

EL OPERATORS

OPERATOR	ALTERNATIVE	#
`== / !=`	`eq` / `ne`	15
`< / >`	`lt` / `gt`	15
`<= / >=`	`le` / `ge`	15
`&& / || / !`	`and` / `or` / `not`	15
`/ and %`	`div` / `mod`	15
`+=`	string concat (EL 3.0)	15

STANDARD ACTIONS

ACTION	DOES	#
`<jsp:include page="x.jsp"/>`	dynamic include	18
`<jsp:forward page="x.jsp"/>`	server-side forward	18
`<jsp:param name="" value=""/>`	pass a parameter	18
`<jsp:useBean id="" class=""/>`	find or create bean	19
`<jsp:setProperty property="*"/>`	fill from params	19
`<jsp:getProperty/>`	print a property	19

⚠ COMMON MISTAKES

- `<%= %>` and `${}` print raw text — escape user data with `<c:out>` or `fn:escapeXml`
- Confusing `<%@ include %>` (compile time) with `<jsp:include>` (request time)
- Business logic in scriptlets instead of a servlet or controller

✓ TIPS & BEST PRACTICES

- Put JSPs under `WEB-INF/` so only your servlets can forward to them
- Prefix links with `${pageContext.request.contextPath}`
- Set `contentType` with `charset=UTF-8` on every page

QUICK SUMMARY

- Directives configure; actions run at request time
- EL `${…}` reads beans, maps, lists and scopes
- Scopes: page → request → session → application
- Use JSTL and EL, not scriptlets

Quick Reference — JSTL & Custom Tags

JSTL SETUP

ITEM	VALUE	#
Core	`jakarta.tags.core` (prefix c)	21
Formatting	`jakarta.tags.fmt` (prefix fmt)	21
Functions	`jakarta.tags.functions` (fn)	21
Old URIs	`http://java.sun.com/jsp/jstl/core`	21
Jar	jakarta.servlet.jsp.jstl + API	21
JSTL 3.0	Jakarta EE 10	21

CORE TAGS

TAG	DOES	#
`<c:out value="${x}"/>`	print, XML-escaped	24
`<c:set var="t" value="…"/>`	set a variable	24
`<c:remove var="t"/>`	remove it	24
`<c:if test="${…}">`	condition (no else)	22
`<c:choose> <c:when>`	if / else-if	22
`<c:otherwise>`	else branch	22
`<c:catch var="e">`	catch exceptions	32

ITERATION & URLS

TAG	DOES	#
`<c:forEach items="${list}" var="x">`	loop a collection	23
`begin / end / step`	numeric loop	23
`varStatus="s"`	`s.index`, `s.first`, `s.last`	23
`<c:forTokens delims=",">`	split a string	23
`<c:url value="/p"/>`	context-relative URL	24
`<c:param name="" value=""/>`	encoded query param	24
`<c:redirect url="…"/>`	send a redirect	24

FMT TAGS

TAG	DOES	#
`<fmt:formatNumber type="currency"/>`	money	25
`<fmt:formatDate pattern="yyyy-MM-dd"/>`	dates	25
`<fmt:parseNumber/> / parseDate`	text → value	25
`<fmt:setLocale value="de_DE"/>`	locale	25
`<fmt:setBundle basename="msg"/>`	resource bundle	25
`<fmt:message key="title"/>`	localized text	25

FN FUNCTIONS

FUNCTION	DOES	#
`fn:length(x)`	size of list / string	26
`fn:contains(s, "a")`	substring test	26
`fn:escapeXml(s)`	escape HTML	26
`fn:substring(s, 0, 5)`	slice	26
`fn:split(s, ",")`	to array	26
`fn:toUpperCase / trim`	transform	26
`fn:join(arr, ", ")`	array to string	26

CUSTOM TAGS

ITEM	DOES	#
`WEB-INF/tags/x.tag`	tag file	27
`<%@ attribute name="" required="true" %>`	tag attribute	27
`<jsp:doBody/>`	render tag body	27
`SimpleTagSupport.doTag()`	modern handler	29
`TagSupport doStartTag`	classic handler	28
`SKIP_BODY / EVAL_BODY_INCLUDE`	classic return codes	28

TLD

ELEMENT	DOES	#
`<taglib> / <uri>`	library + its URI	30
`<tag><name>`	tag name	30
`<tag-class>`	handler class	30
`<body-content>`	empty / scriptless / tagdependent	30
`<attribute><rtexprvalue>`	allow EL values	30
`<function>`	EL function mapping	17

MVC & ERRORS

ITEM	DOES	#
`req.setAttribute("m", m)`	servlet → JSP data	31
`getRequestDispatcher(…).forward`	render the JSP	31
`sendRedirect`	PRG after POST	31
`<error-page> in web.xml`	map codes / exceptions	32
`isErrorPage="true"`	get `exception`	32
`${pageContext.errorData.statusCode}`	error info	32

⚠ COMMON MISTAKES

- Old `java.sun.com` taglib URIs on a Jakarta EE 10+ server — use `jakarta.tags.*`
- `<c:if>` has no else — reach for `<c:choose>`
- Printing user input with `${}` instead of `<c:out>` (XSS)

✓ TIPS & BEST PRACTICES

- Wrap repeated markup in a `.tag` file instead of copy-paste
- Use `varStatus` for row striping and "last item" commas
- Forward from a servlet; let the JSP only render

QUICK SUMMARY

- `c:` control flow and output, `fmt:` i18n, `fn:` strings
- `<c:forEach>` + `varStatus` for loops
- Tag files for reuse; handlers + TLD for logic
- Servlet sets attributes, JSP renders them